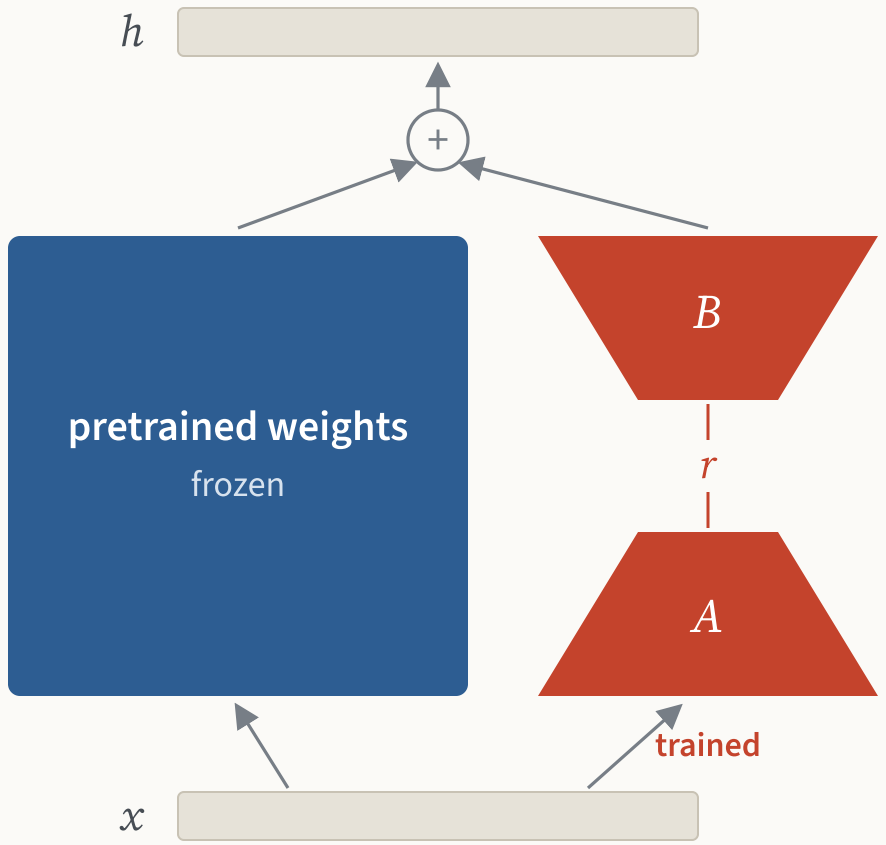
Low-Rank Adaptation

Teach a frozen model a new task by learning a small, low-rank change to its weights.



Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen
Microsoft. ICLR 2022. Results on images from Zhang, Zhou & Liu 2022 and Zhu et al. 2024.

Presented by **Ertugrul Senturk**
October 2026

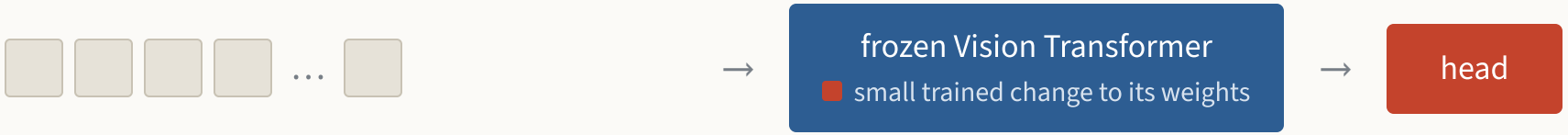
From VPT to LoRA

VPT
last time



Learned vectors join the image patches, so every image carries a longer sequence through every layer.

LoRA
today



The input stays as it is. **A small change to the weights** is learned, and after training it is added into them.

VPT trained 0.6% of the weights and beat full fine-tuning on 20 of 24 tasks. But its prompts make every image slower, and they can never be folded into the model.

LoRA leaves the input alone and changes the weights instead. It ends up storing about as little per task as VPT, while the model runs exactly as fast as before.

VPT: Jia et al., ECCV 2022 (0.6% of the weights with 50 prompts per layer and a 102-class head; 20 of 24 tasks, Table 1). LoRA: Hu et al., ICLR 2022, first posted in June 2021.

Every fine-tuned copy is as big as the model

The paper starts from the largest model of its time, GPT-3, with 175 billion weights. Each fine-tuned copy takes 350 GB, so a service with a hundred tasks would store 35 TB. LoRA keeps one shared model and only a small file per task, and the frozen weights also need no optimizer state, which cuts training memory from 1.2 TB to 350 GB.

Shown for **1 task** on **ViT-B/16, 86 million weights**. The next pages show 100 tasks, then GPT-3.

Full fine-tuning: 1 retrained ViT-B copy



343 MB

1 copy at 343 MB each

LoRA: one shared ViT-B plus 1 small file



each red speck is one task file, to scale

345 MB

343 MB once, plus 1.5 MB per task

GPT-3 in 16-bit numbers: 350 GB per copy, 35 MB per LoRA file at rank 4 on the query and value weights (Hu et al. 2022, Sec. 4.2). ViT-B/16 in 32-bit: 343 MB per copy; its LoRA file is rank 8 on the same weights plus a 102-class head. Squares are sized by area.

Every fine-tuned copy is as big as the model

The paper starts from the largest model of its time, GPT-3, with 175 billion weights. Each fine-tuned copy takes 350 GB, so a service with a hundred tasks would store 35 TB. LoRA keeps one shared model and only a small file per task, and the frozen weights also need no optimizer state, which cuts training memory from 1.2 TB to 350 GB.

Shown for **100 tasks** on **ViT-B/16, 86 million weights**.

Full fine-tuning: 100 retrained ViT-B copies



34 GB

100 copies at 343 MB each

LoRA: one shared ViT-B plus 100 small files



493 MB

343 MB once, plus 1.5 MB per task

GPT-3 in 16-bit numbers: 350 GB per copy, 35 MB per LoRA file at rank 4 on the query and value weights (Hu et al. 2022, Sec. 4.2). ViT-B/16 in 32-bit: 343 MB per copy; its LoRA file is rank 8 on the same weights plus a 102-class head. Squares are sized by area.

Every fine-tuned copy is as big as the model

The paper starts from the largest model of its time, GPT-3, with 175 billion weights. Each fine-tuned copy takes 350 GB, so a service with a hundred tasks would store 35 TB. LoRA keeps one shared model and only a small file per task, and the frozen weights also need no optimizer state, which cuts training memory from 1.2 TB to 350 GB.

Shown for **100 tasks** on **GPT-3, 175 billion weights**.

Full fine-tuning: 100 retrained GPT-3 copies



35 TB

100 copies at 350 GB each

LoRA: one shared GPT-3 plus 100 small files

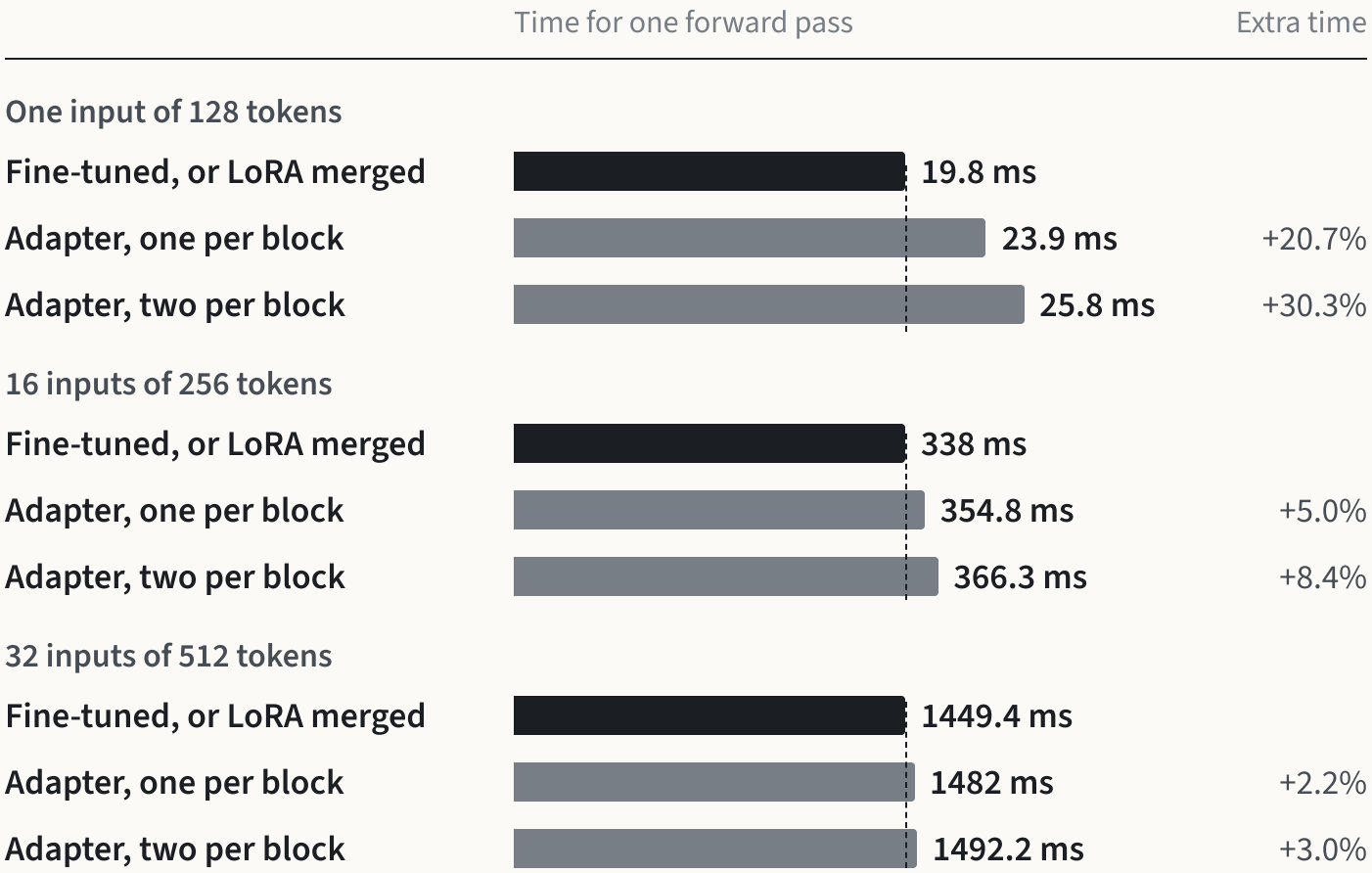


354 GB

350 GB once, plus 35 MB per task

GPT-3 in 16-bit numbers: 350 GB per copy, 35 MB per LoRA file at rank 4 on the query and value weights (Hu et al. 2022, Sec. 4.2). ViT-B/16 in 32-bit: 343 MB per copy; its LoRA file is rank 8 on the same weights plus a 102-class head. Squares are sized by area.

Earlier cheap methods cost time or sequence length



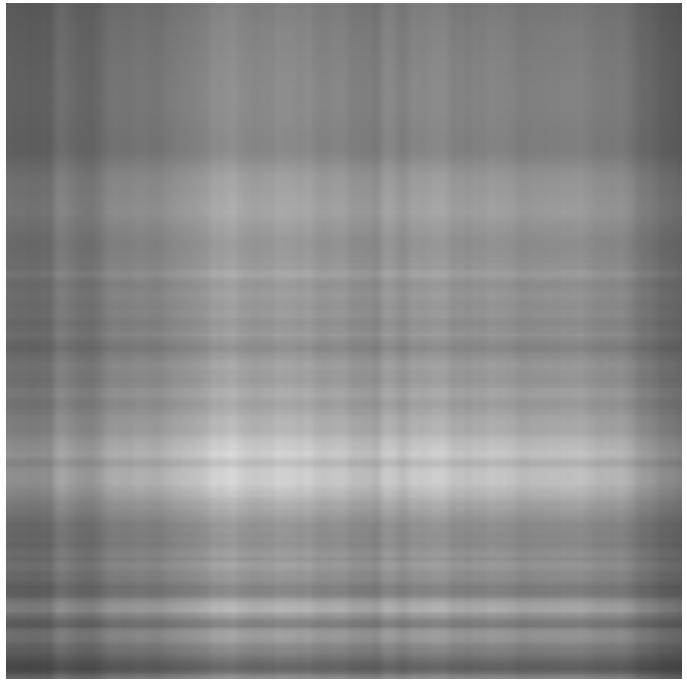
Adapters insert small extra layers into every block. They have few weights, but an image or a sentence must pass through them one after another, so serving a single input takes 20 to 30% longer. With large batches the gap shrinks to 2 or 3%.

Prompt methods such as prefix tuning and VPT lengthen the input instead. That leaves less room for the real input and costs compute in every layer, and on GPT-3 prefix tuning got worse beyond 256 prompt tokens.

LoRA adds no layer and no token at test time.

Hu et al. 2022, Table 1: one forward pass of GPT-2 medium, mean of 100 trials on one NVIDIA Quadro RTX 8000; bars are drawn relative to the fine-tuned model in each setting. Prefix tuning: Sec. 5.5 and Table 15.

What a low-rank matrix is



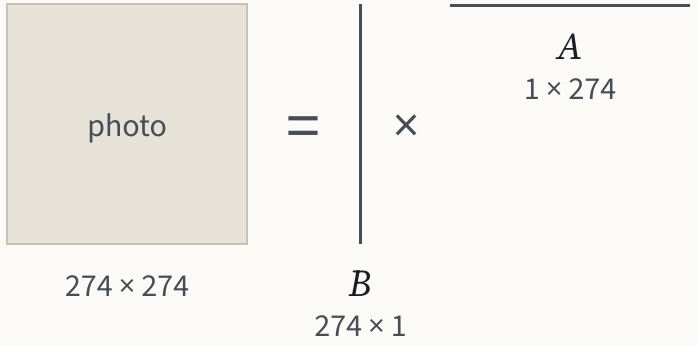
Rank 1: 1 × (274 + 274) = **548** numbers, 0.7% of the photo's 75,076

1	2	4	8	16	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

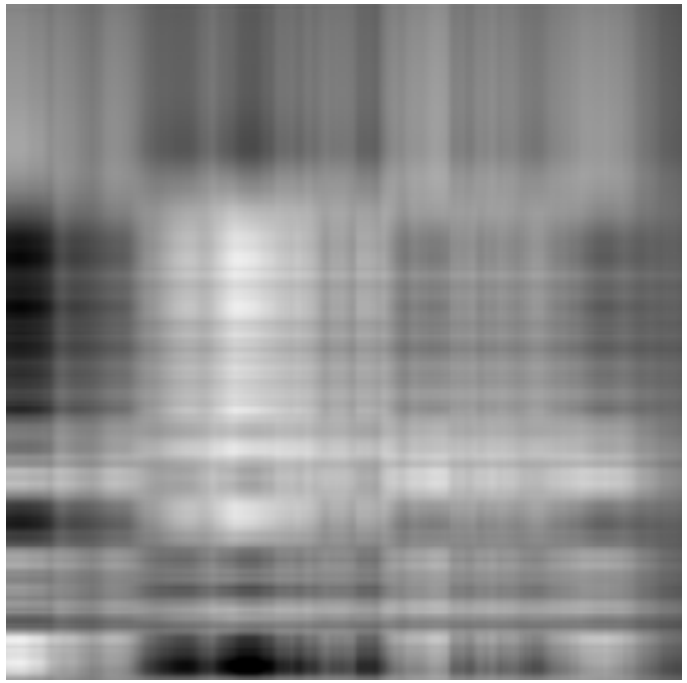
One piece gives only bands of light and dark, because every row of it is the same pattern, scaled.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



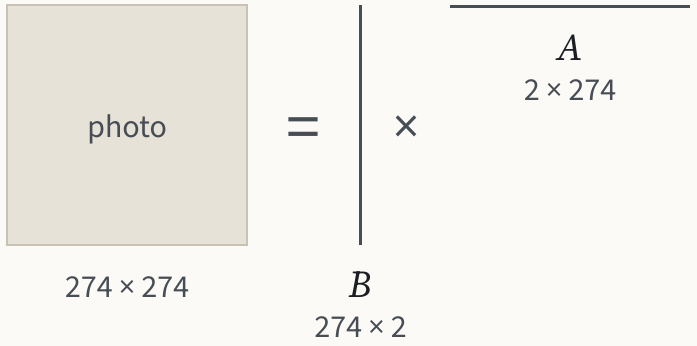
Rank 2: 2 × (274 + 274) = **1,096** numbers, 1.5% of the photo's 75,076

1	2	4	8	16	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

Two pieces are still mostly bands, now in two patterns.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



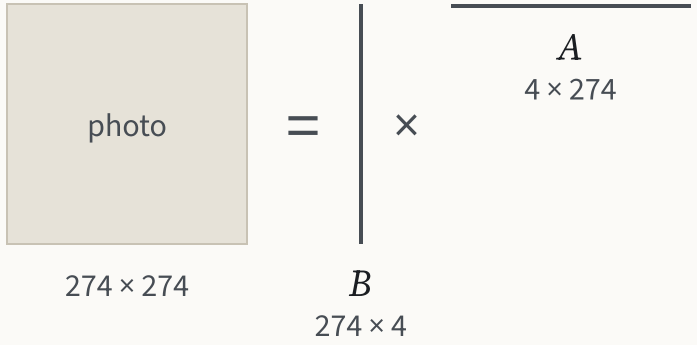
Rank 4: 4 × (274 + 274) = **2,192** numbers, 2.9% of the photo's 75,076

1	2	**4**	8	16	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

With four pieces the main light and dark areas appear.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



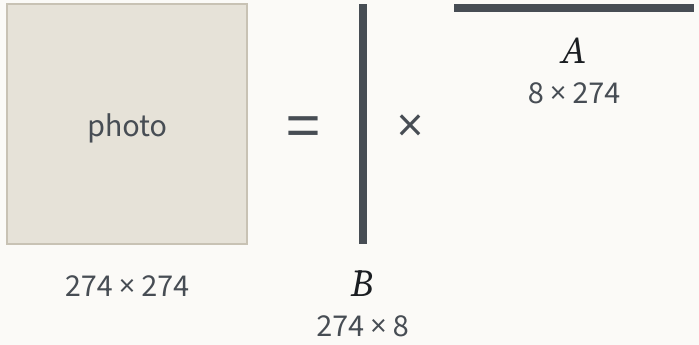
Rank 8: 8 × (274 + 274) = **4,384** numbers, 5.8% of the photo's 75,076

1	2	4	8	16	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

With eight pieces, 6% of the numbers, the rough shape of the cat appears, because the first pieces carry most of the picture.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



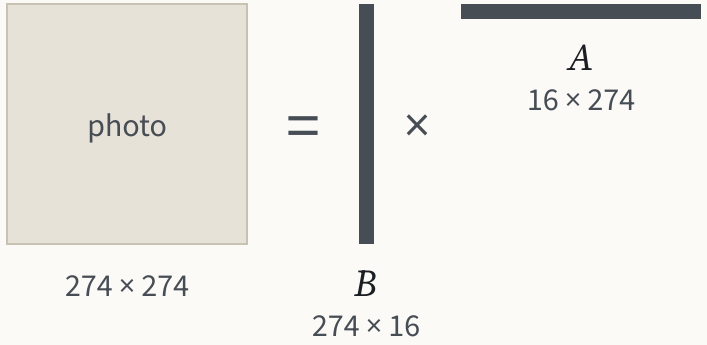
Rank 16: 16 × (274 + 274) = **8,768** numbers,
11.7% of the photo's 75,076

1	2	4	8	**16**	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

With 16 pieces the cat is recognisable, from 12% of the numbers.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



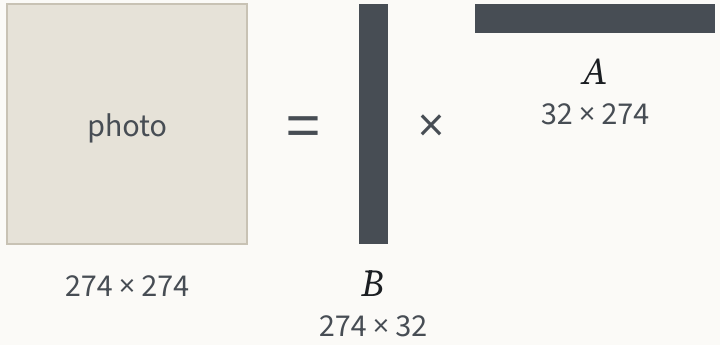
Rank 32: 32 × (274 + 274) = **17,536** numbers,
23.4% of the photo's 75,076

1	2	4	8	16	**32**	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

With 32 pieces the face and the fur appear.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



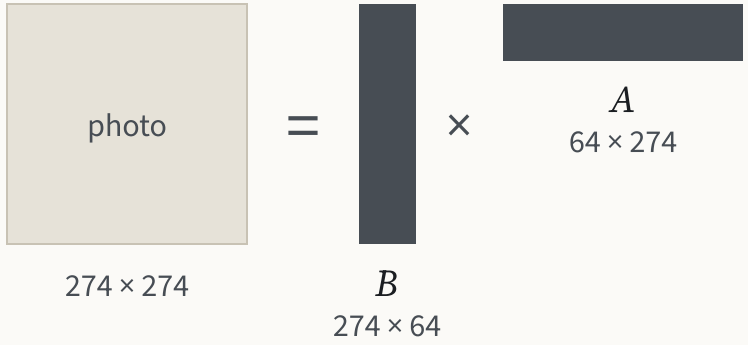
Rank 64: 64 × (274 + 274) = **35,072** numbers, 46.7% of the photo's 75,076

1	2	4	8	16	32	**64**	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

With 64 pieces it is hard to tell from the original.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

What a low-rank matrix is



75,076 numbers: the full photo

1	2	4	8	16	32	64	all 274

A grayscale photo is a matrix of 274 × 274 brightness values. Any matrix can be written as a sum of simple pieces, where each piece is one column of numbers times one row. The rank is the number of pieces.

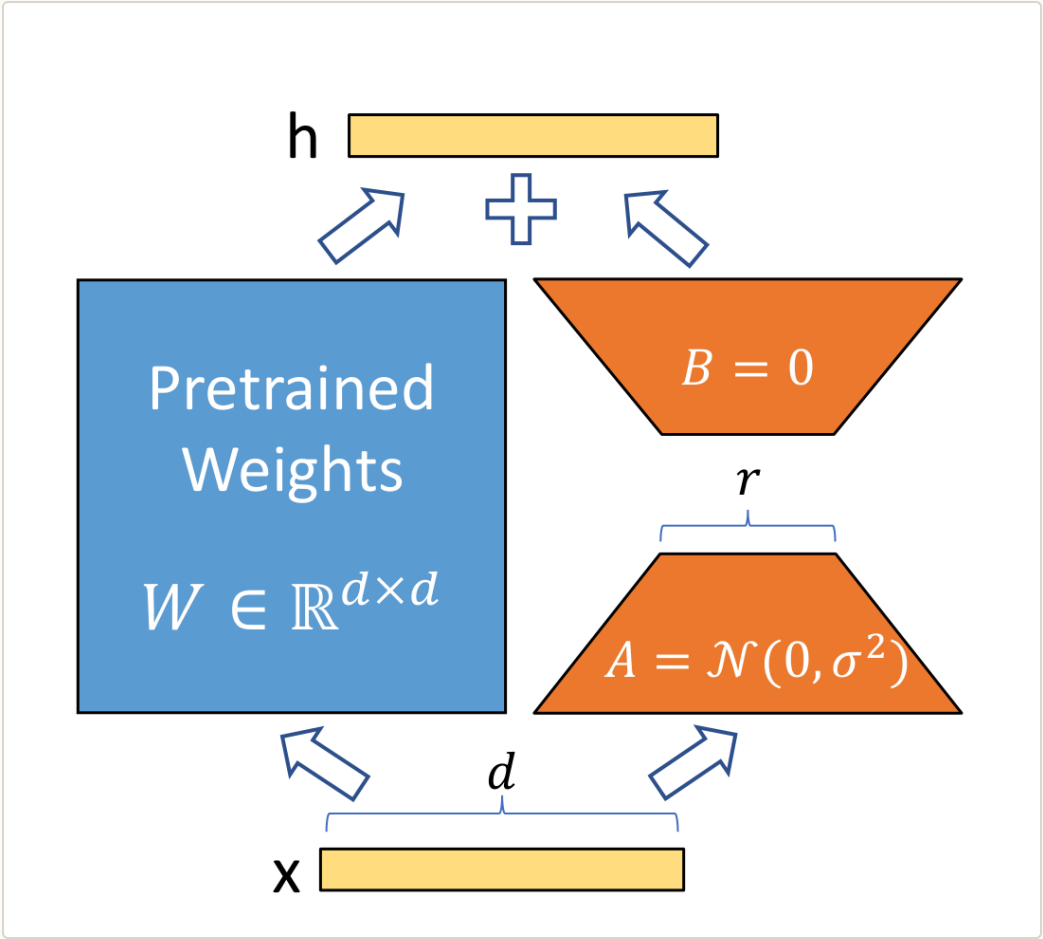
All 274 pieces give back the photo exactly.

> LoRA bets that the change fine-tuning makes to a weight matrix is like this photo: a few pieces carry almost all of it.



Not from the paper: the best rank-r approximation (SVD) of an Oxford Pets photo, Jia et al. 2022, Fig. 10. The low-rank bet follows Li et al. 2018 and Aghajanyan et al. 2021.

LoRA adds a trainable bypass to a frozen weight



During training | After training

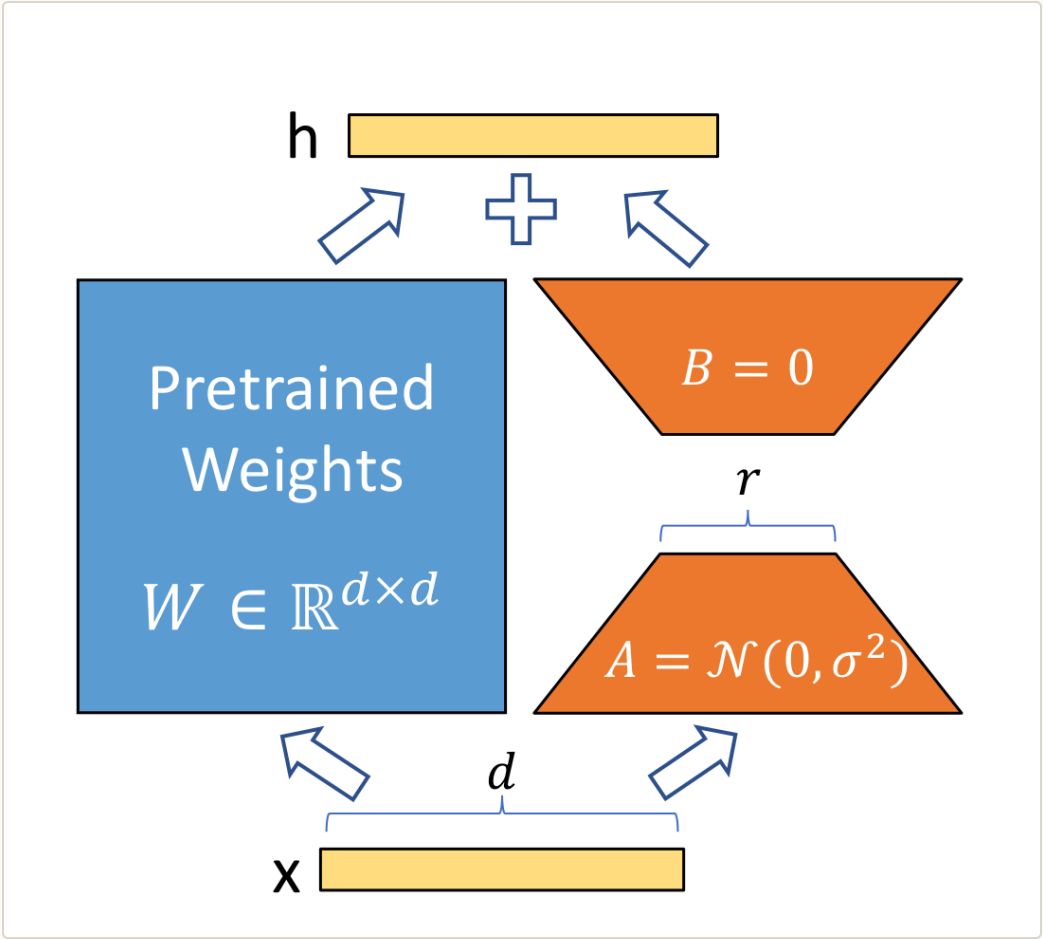
The input goes through the frozen weight W_0 and, in parallel, through A and B, which are trained. The two results are added. The full change ΔW is never built; LoRA trains the thin matrices directly, with a rank r chosen by hand.

$$h = W_0 x + (\alpha / r)\, BAx$$

B starts at zero, so training begins exactly at the pretrained model. Only A and B get gradients and optimizer state.

Figure: Hu et al. 2022, Fig. 1, where orange is trained. Equation (3) and Sec. 4.1. The paper sets α to the first r it tries and does not tune it.

LoRA adds a trainable bypass to a frozen weight



During training | **After training**

After training, the product BA is added into the weight once. The model is again one matrix per layer, of the same size, so it runs exactly as fast as before.

$$W = W_0 + (\alpha/r)\, BA$$

$$h = Wx$$

To switch task, subtract this BA and add that task's $B'A'$. Per task, only A and B are stored.

Figure: Hu et al. 2022, Fig. 1, where orange is trained. Equation (3) and Sec. 4.1. The paper sets α to the first r it tries and does not tune it.

LoRA on a 3 × 3 weight matrix

Frozen path | LoRA path | Merging | Merged layer

During training

$$\underbrace{\begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 0 \\ 0 & 1 & 2 \end{bmatrix}}_{W} \underbrace{\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 5 \\ 7 \\ 8 \end{bmatrix}}_{Wx}$$

After training, once

At test time

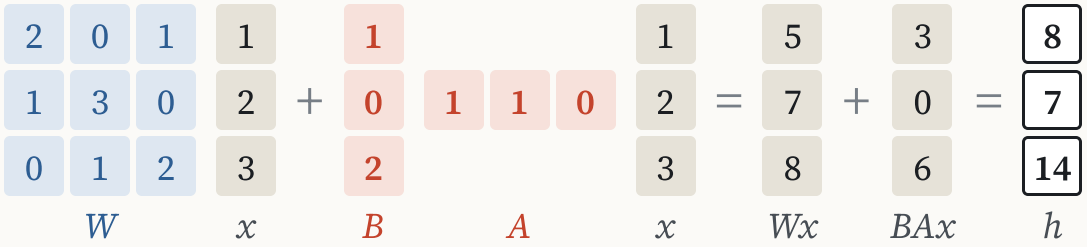
Take a tiny weight W and an input x. The frozen path is just the pretrained layer, and Wx gives 5, 7 and 8.

Made-up numbers with rank $r = 1$ and $\alpha/r = 1$, so every product can be checked by eye. In ViT-B the same happens with 768 × 768 weights and $r = 8$.

LoRA on a 3 × 3 weight matrix

Frozen path | **LoRA path** | Merging | Merged layer

During training



After training, once

At test time

LoRA adds a second path. A squeezes the input to r numbers, here a single 3, and B expands it back to three; the two paths are added. In training, only the six red numbers change.

Made-up numbers with rank $r = 1$ and $\alpha/r = 1$, so every product can be checked by eye. In ViT-B the same happens with 768 × 768 weights and $r = 8$.

LoRA on a 3 × 3 weight matrix

Frozen path | LoRA path | **Merging** | Merged layer

During training

$$\underset{W}{\begin{bmatrix}2&0&1\\1&3&0\\0&1&2\end{bmatrix}}\underset{x}{\begin{bmatrix}1\\2\\3\end{bmatrix}}+\underset{B}{\begin{bmatrix}1\\0\\2\end{bmatrix}}\underset{A}{\begin{bmatrix}1&1&0\end{bmatrix}}\underset{x}{\begin{bmatrix}1\\2\\3\end{bmatrix}}=\underset{Wx}{\begin{bmatrix}5\\7\\8\end{bmatrix}}+\underset{BAx}{\begin{bmatrix}3\\0\\6\end{bmatrix}}=\underset{h}{\boxed{\begin{bmatrix}8\\7\\14\end{bmatrix}}}$$

After training, once

$$\underset{B}{\begin{bmatrix}1\\0\\2\end{bmatrix}}\underset{A}{\begin{bmatrix}1&1&0\end{bmatrix}}=\underset{BA}{\begin{bmatrix}1&1&0\\0&0&0\\2&2&0\end{bmatrix}}\qquad\underset{W}{\begin{bmatrix}2&0&1\\1&3&0\\0&1&2\end{bmatrix}}+\underset{BA}{\begin{bmatrix}1&1&0\\0&0&0\\2&2&0\end{bmatrix}}=\underset{W'}{\begin{bmatrix}3&1&1\\1&3&0\\2&3&2\end{bmatrix}}$$

At test time

After training, B times A is worked out once. A column times a row is a full 3 × 3 matrix, the same shape as W, so it is simply added to it; the red cells are the ones that changed.

Made-up numbers with rank $r = 1$ and $\alpha/r = 1$, so every product can be checked by eye. In ViT-B the same happens with 768 × 768 weights and $r = 8$.

LoRA on a 3 × 3 weight matrix

Frozen path | LoRA path | Merging | **Merged layer**

During training

$$\underset{W}{\begin{bmatrix}2&0&1\\1&3&0\\0&1&2\end{bmatrix}}\underset{x}{\begin{bmatrix}1\\2\\3\end{bmatrix}}+\underset{B}{\begin{bmatrix}1\\0\\2\end{bmatrix}}\underset{A}{\begin{bmatrix}1&1&0\end{bmatrix}}\underset{x}{\begin{bmatrix}1\\2\\3\end{bmatrix}}=\underset{Wx}{\begin{bmatrix}5\\7\\8\end{bmatrix}}+\underset{BAx}{\begin{bmatrix}3\\0\\6\end{bmatrix}}=\underset{h}{\boxed{\begin{bmatrix}8\\7\\14\end{bmatrix}}}$$

After training, once

$$\underset{B}{\begin{bmatrix}1\\0\\2\end{bmatrix}}\underset{A}{\begin{bmatrix}1&1&0\end{bmatrix}}=\underset{BA}{\begin{bmatrix}1&1&0\\0&0&0\\2&2&0\end{bmatrix}}\qquad\underset{W}{\begin{bmatrix}2&0&1\\1&3&0\\0&1&2\end{bmatrix}}+\underset{BA}{\begin{bmatrix}1&1&0\\0&0&0\\2&2&0\end{bmatrix}}=\underset{W'}{\begin{bmatrix}3&1&1\\1&3&0\\2&3&2\end{bmatrix}}$$

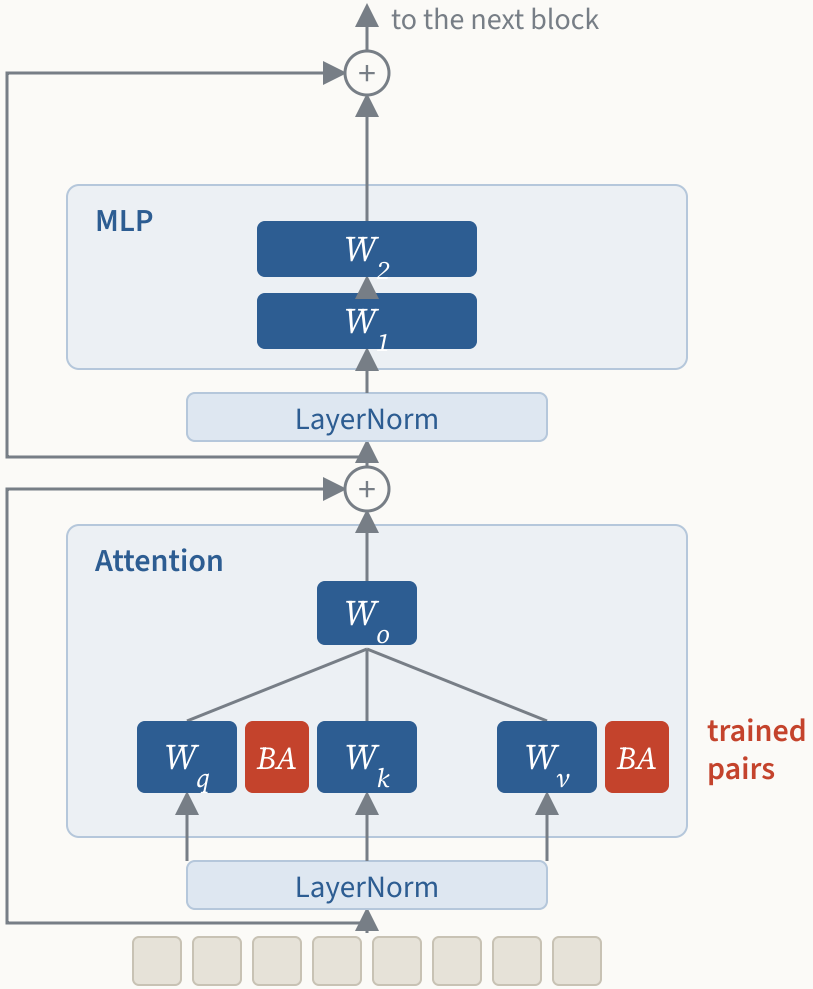
At test time

$$\underset{W'}{\begin{bmatrix}3&1&1\\1&3&0\\2&3&2\end{bmatrix}}\underset{x}{\begin{bmatrix}1\\2\\3\end{bmatrix}}=\underset{h}{\boxed{\begin{bmatrix}8\\7\\14\end{bmatrix}}}$$

The merged weight gives exactly the same output, because $Wx + B(Ax) = (W + BA)x$. B and A are deleted, and the layer is one ordinary matrix again.

Made-up numbers with rank $r = 1$ and $\alpha/r = 1$, so every product can be checked by eye. In ViT-B the same happens with 768 × 768 weights and $r = 8$.

Where LoRA goes in a Vision Transformer

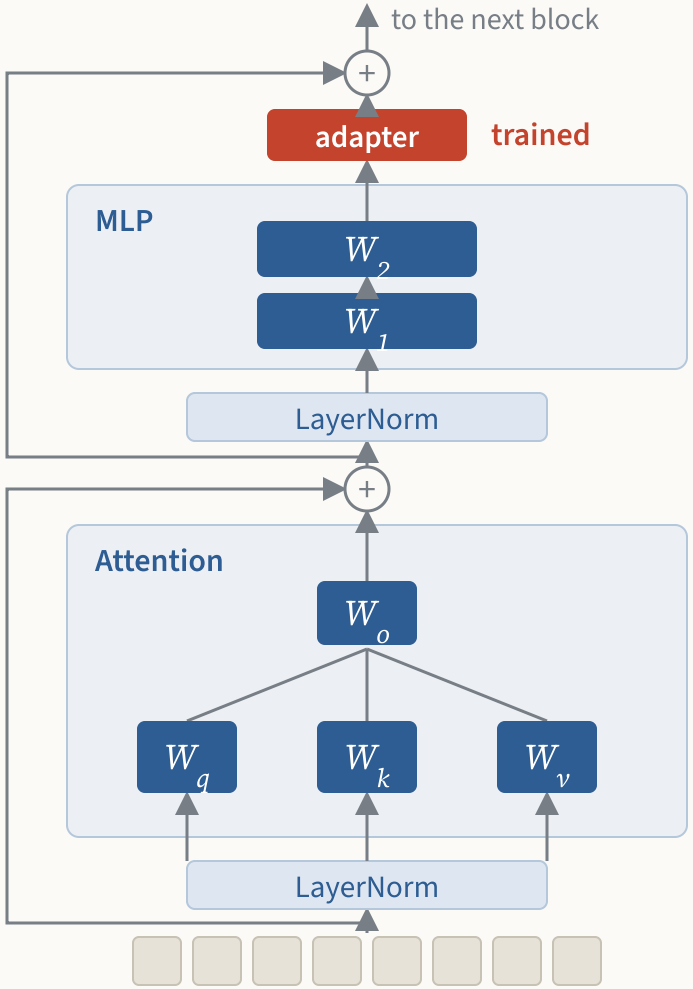


LoRA | Adapter

LoRA puts a trained pair $B\ A$ beside the query and value weights of the attention, in all 12 blocks. The key and output weights, the MLP and the layer norms stay frozen. After merging, the block is exactly the original.

One of the 12 blocks of ViT-B/16. Placement as in Hu et al. 2022, Sec. 4.2, and Houlsby et al. 2019. Zhang et al. 2022, Fig. 1a, draws LoRA, the adapter and VPT in one block.

Where LoRA goes in a Vision Transformer



LoRA | Adapter

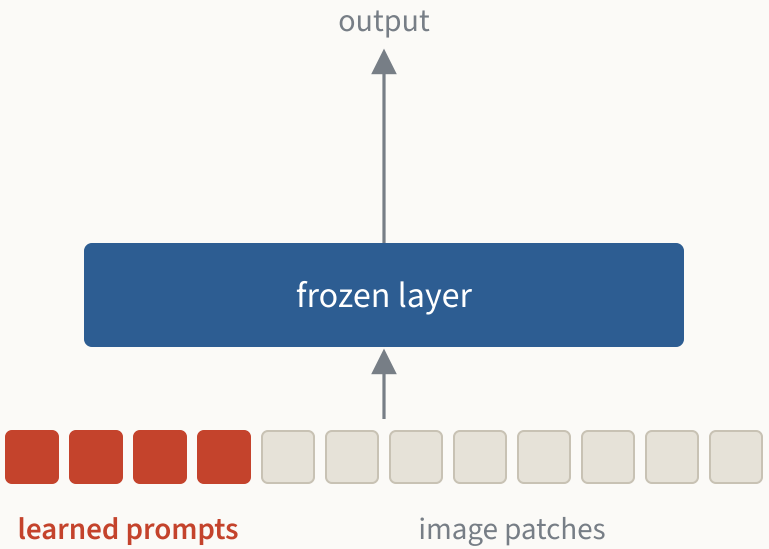
An adapter, for contrast, is a small trained two-layer network inserted after the MLP. It is an extra layer that every input must pass through, which is where its extra time comes from; LoRA sits beside weights that already exist.

One of the 12 blocks of ViT-B/16. Placement as in Hu et al. 2022, Sec. 4.2, and Houlsby et al. 2019. Zhang et al. 2022, Fig. 1a, draws LoRA, the adapter and VPT in one block.

VPT and LoRA side by side

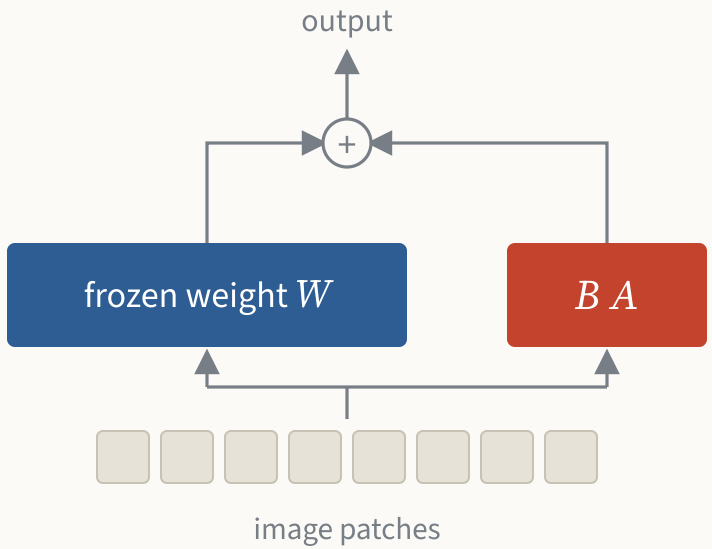
What is learned	During training	At test time

VPT



VPT learns extra input vectors. The same 50 per layer are placed in front of every image of the task; they start random and are not taken from any image.

LoRA



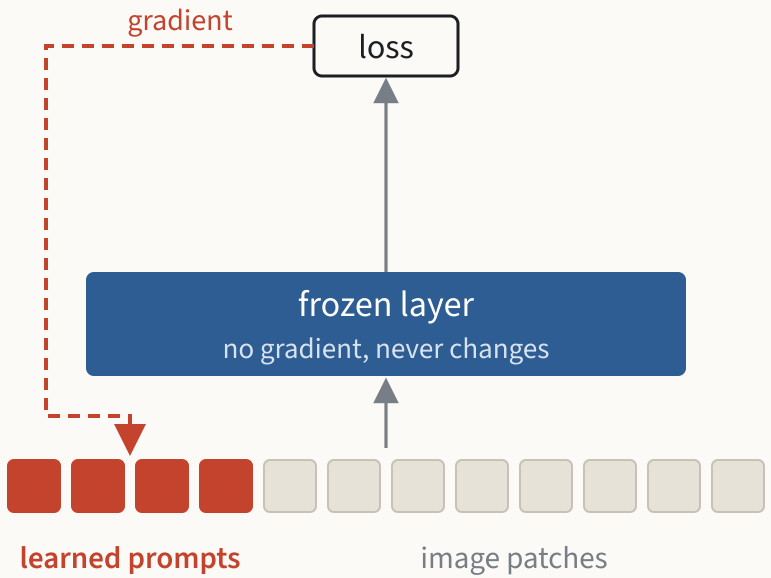
LoRA learns a correction to the weights, the pair B and A beside the query and value weights. It is also one pair for the whole task, and B starts at zero.

One layer of ViT-B/16 drawn for each. VPT-deep with 50 prompts per layer (Jia et al. 2022); LoRA with rank 8 on the query and value weights (Hu et al. 2022). Both train the same 102-class head.

VPT and LoRA side by side

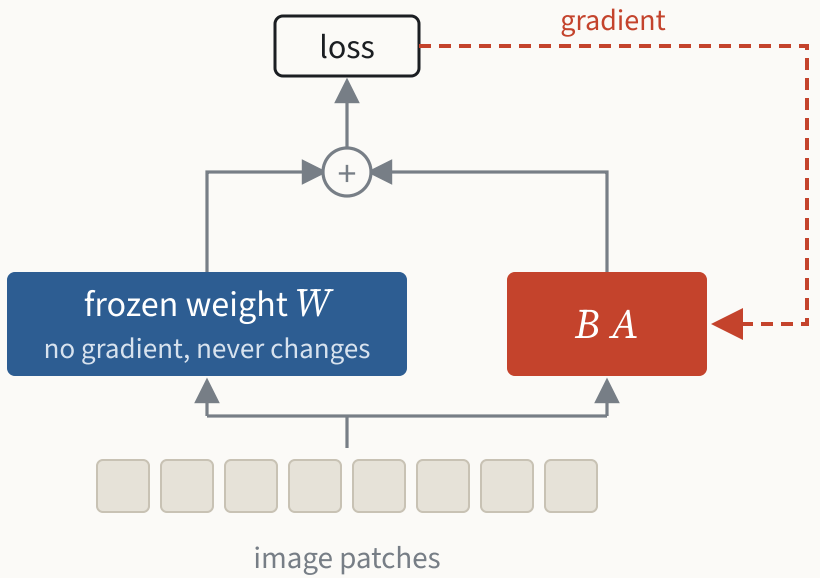
What is learned | During training | At test time

VPT



Each training image goes through the frozen model with the prompts attached. The loss sends gradients back, and only the prompts and the head change.

LoRA



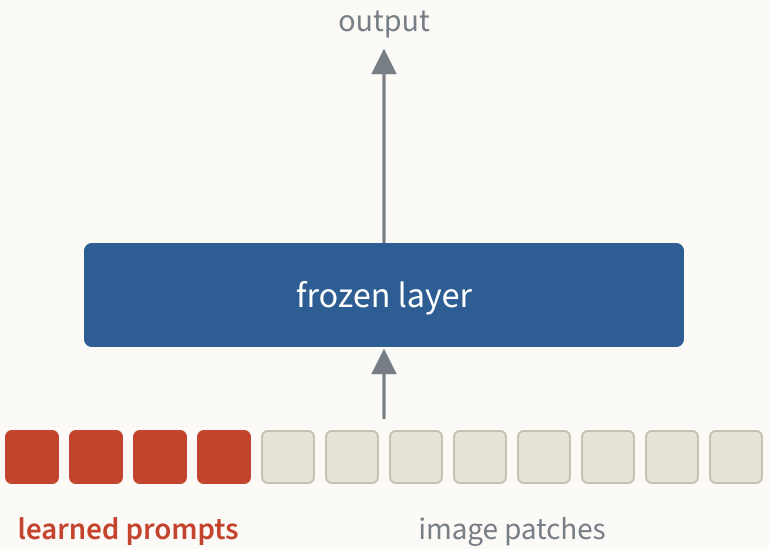
Each training image takes both paths, Wx and BAx. Only B, A and the head change; W never receives a gradient.

One layer of ViT-B/16 drawn for each. VPT-deep with 50 prompts per layer (Jia et al. 2022); LoRA with rank 8 on the query and value weights (Hu et al. 2022). Both train the same 102-class head.

VPT and LoRA side by side

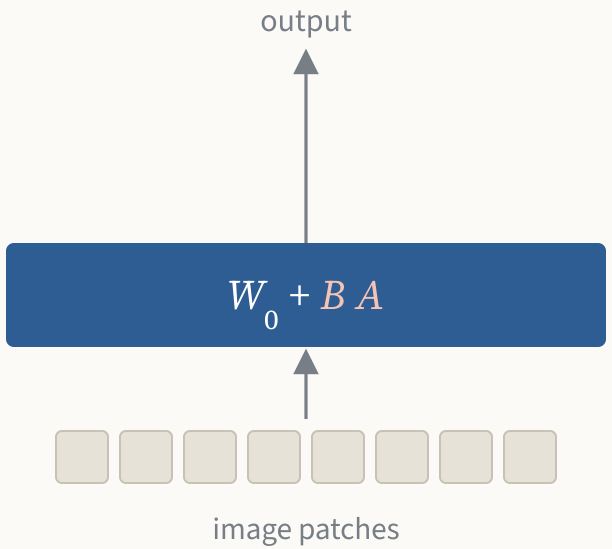
What is learned	During training	At test time

VPT



The prompts must be attached to every test image, so each layer works on 247 tokens instead of 197. They are inputs, so they can never be folded into the weights.

LoRA



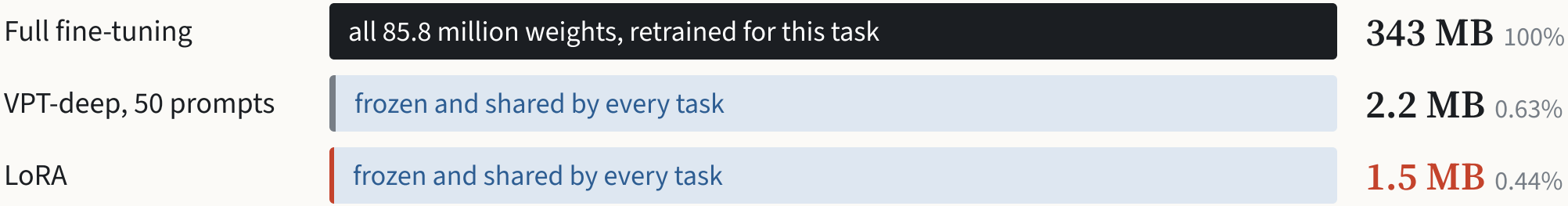
BA is added into W once, and the layer is one plain matrix again. The test image goes through exactly the original model.

One layer of ViT-B/16 drawn for each. VPT-deep with 50 prompts per layer (Jia et al. 2022); LoRA with rank 8 on the query and value weights (Hu et al. 2022). Both train the same 102-class head.

How much LoRA trains and stores per task

$2 \cdot N \cdot m \cdot d \cdot r = 2 \times 12 \times 2 \times 768 \times 8 =$ **294,912** numbers in the pairs, plus 78,438 for a 102-class head

Numbers trained and stored per task



Compute per image at test time

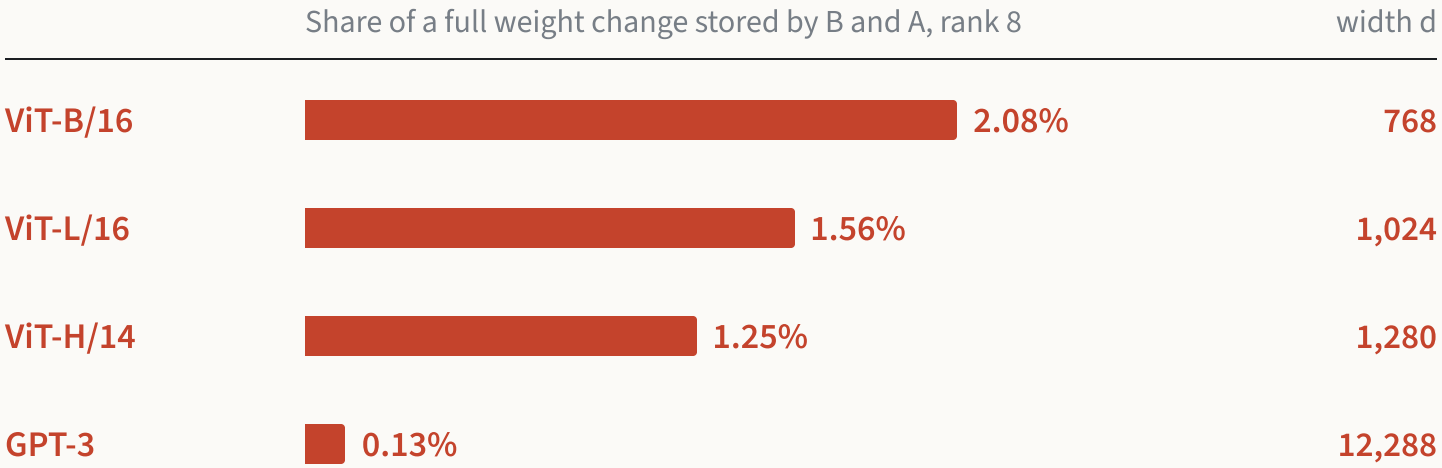


Shown for rank 8 on the query and value weights. At rank 64 on all four attention weights the file grows to 19 MB, 5.6% of the model, and the compute per image does not change.

ViT-B/16: $N = 12$ blocks, $d = 768$, m adapted weights per block, 32-bit numbers. VPT-deep as last time. Compute counts multiply-adds per layer for n tokens ($12nd^2 + 2n^2d$).

LoRA's saving grows with the width of the model

Full change ΔW	$d \times d = d^2$	589,824 numbers on ViT-B
B and A together	$d \times r + r \times d = 2rd$	12,288 numbers on ViT-B



A full change to one $d \times d$ weight has d^2 numbers, while B and A together have $2rd$. Since d is the square root of d^2, LoRA's cost grows like the square root of the full size, times the small constant $2r$.

So the wider the model, the smaller LoRA's share. Together with adapting only the query and value weights, this is why GPT-3's task file is 10,000 times smaller than a copy.

One $d \times d$ attention weight, rank $r = 8$ for every model. Widths: ViT-B/16 768, ViT-L/16 1,024, ViT-H/14 1,280 (Dosovitskiy et al. 2021); GPT-3 12,288 (Hu et al. 2022, Sec. 1). 10,000 times: Hu et al. 2022, Sec. 4.2.

LoRA on a ViT in a few lines

```
vit = timm.create_model(BACKBONE, pretrained=True, num_classes=C)
vit.requires_grad_(False); vit.head.requires_grad_(True)

class LoRAqv(nn.Module):                # wraps qkv = Linear(768, 2304)
    def __init__(self, qkv, r=8, alpha=8):
        super().__init__()
        d = qkv.in_features
        self.qkv, self.s = qkv, alpha / r    # frozen W0, scale α/r
        self.Aq = nn.Parameter(torch.randn(r, d) / d ** 0.5)
        self.Av = nn.Parameter(torch.randn(r, d) / d ** 0.5)
        self.Bq = nn.Parameter(torch.zeros(d, r))   # B = 0
        self.Bv = nn.Parameter(torch.zeros(d, r))
    def forward(self, x):
        q, k, v = self.qkv(x).chunk(3, dim=-1)      # W0 x
        q = q + self.s * x @ self.Aq.T @ self.Bq.T  # + B A x
        v = v + self.s * x @ self.Av.T @ self.Bv.T
        return torch.cat([q, k, v], dim=-1)

for blk in vit.blocks: blk.attn.qkv = LoRAqv(blk.attn.qkv)
```

B starts at zero and A at small random values, so the first forward pass is exactly the pretrained model.

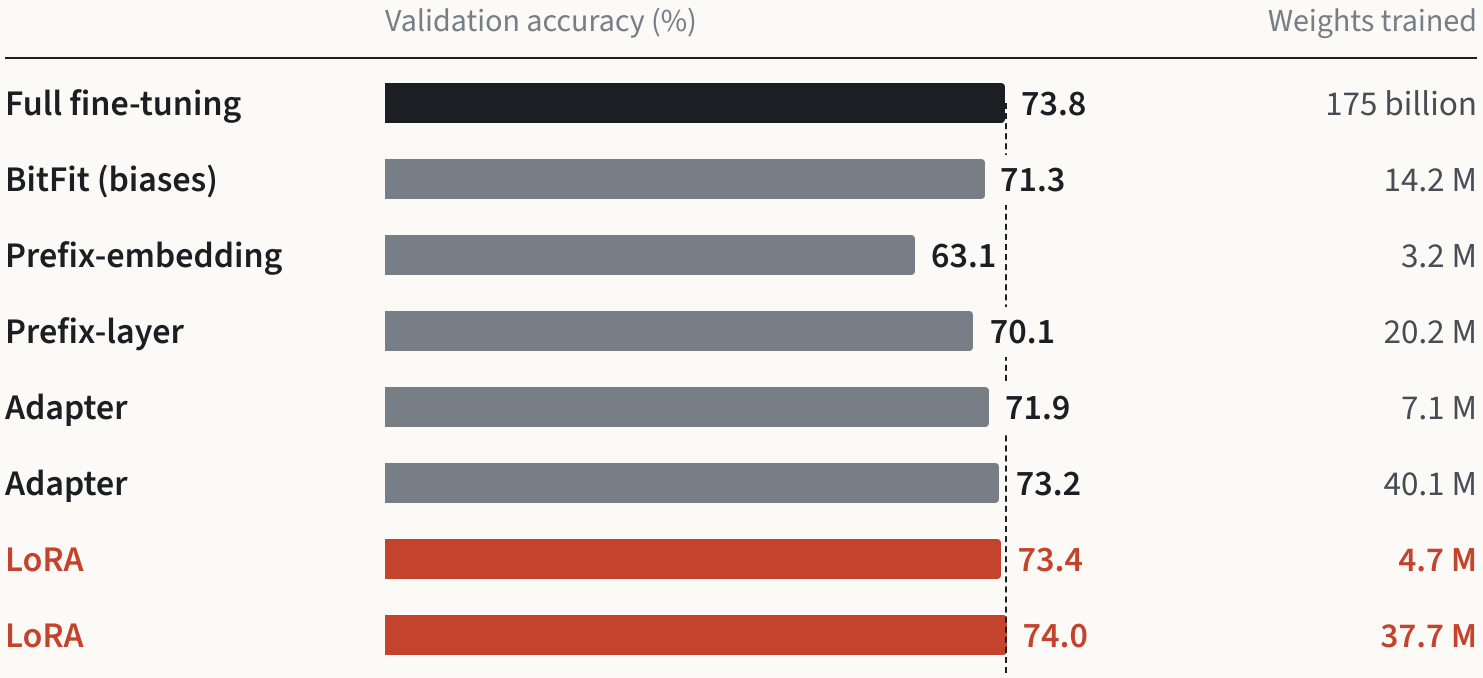
The two new choices are the rank r and which weights get a pair. On ViT-B, Zhang et al. use rank 8 with AdamW at a learning rate of 0.001 for 100 epochs.

After training, each pair is added into its qkv weight once, and the model is a plain timm ViT again.

My sketch for timm's ViT-B/16: 294,912 trained numbers in the pairs, and identical outputs before and after merging. Official code: loralib, github.com/microsoft/LoRA. Settings: Hu et al. 2022, Sec. 4.1; Zhang et al. 2022, App. A.2.

Results in the paper: language models

GPT-3, WikiSQL | GPT-3, MNLI | RoBERTa-base, GLUE



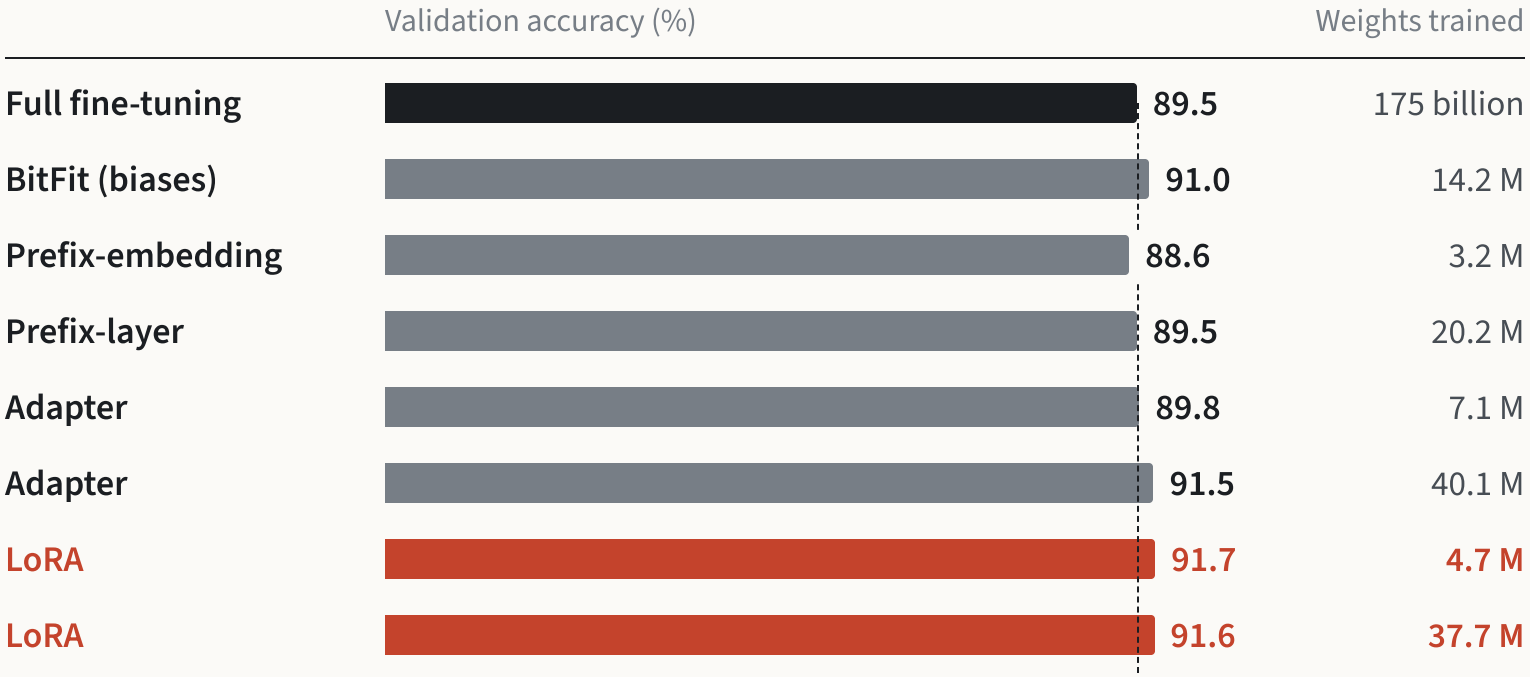
WikiSQL turns a question into an SQL query. On GPT-3, LoRA with 4.7 million trained weights, 0.003% of the model, is within the run-to-run noise of full fine-tuning, and slightly above it with 37.7 million.

The prefix methods are far behind, and the adapter needs more weights to come close.

Hu et al. 2022, Table 4: GPT-3 175B, validation accuracy, noise about ±0.5. Adapter: two per block (Houlsby et al.). BitFit trains only the biases.

Results in the paper: language models

GPT-3, WikiSQL | **GPT-3, MNLI** | RoBERTa-base, GLUE

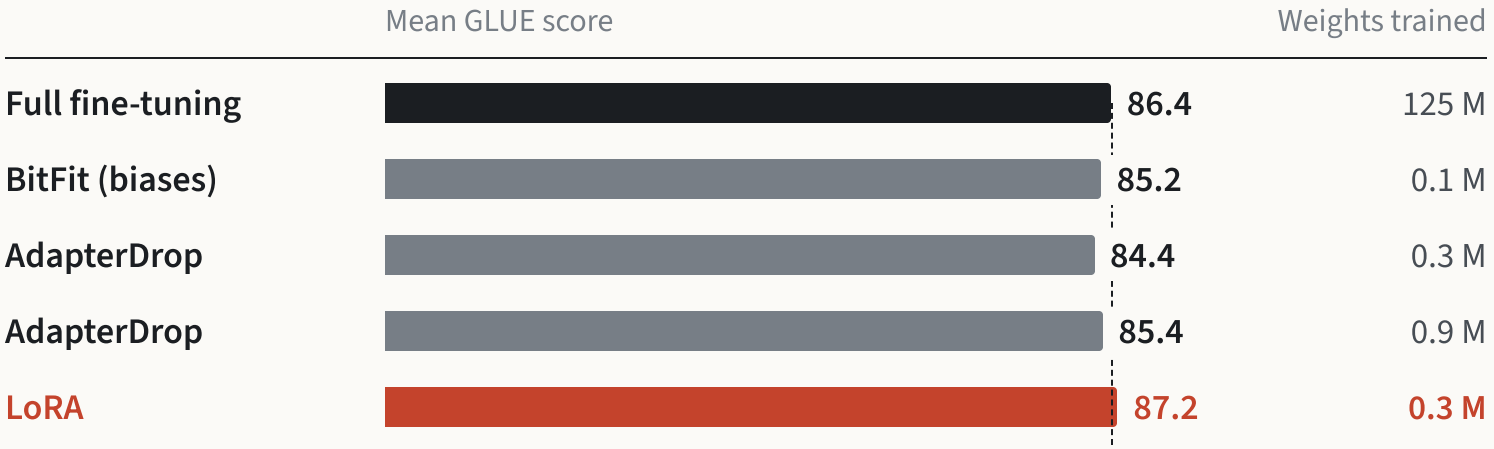


MNLI asks whether one sentence follows from another. Here LoRA is two points above full fine-tuning, 91.7 against 89.5, and above every other cheap method.

Hu et al. 2022, Table 4: GPT-3 175B, validation accuracy on MNLI-matched, noise about ±0.1. Adapter: two per block (Houlsby et al.). BitFit trains only the biases.

Results in the paper: language models

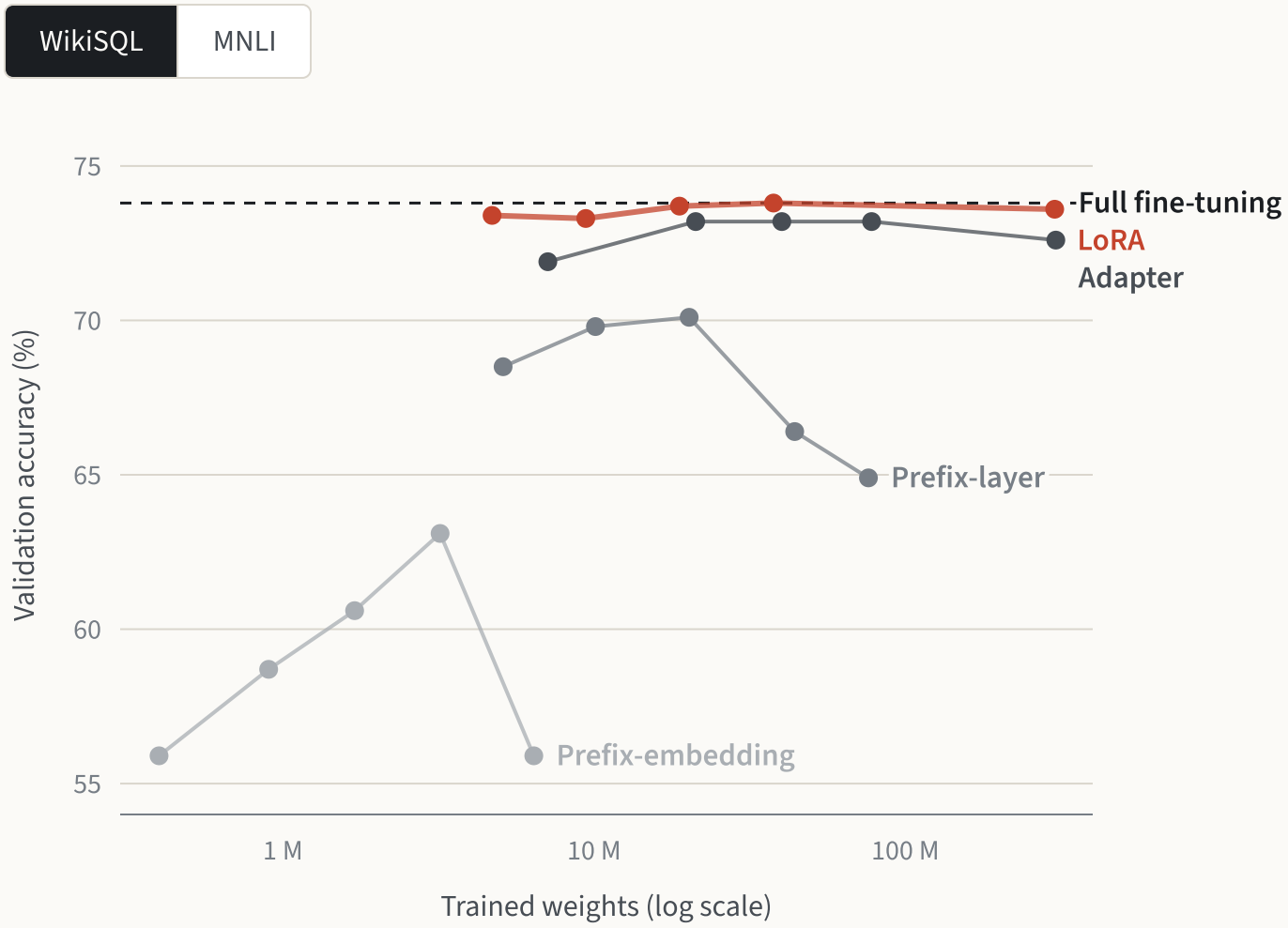
GPT-3, WikiSQL | GPT-3, MNLI | **RoBERTa-base, GLUE**



RoBERTa-base has 125 million weights, close to ViT-B. Averaged over the eight GLUE language tasks, LoRA with 0.3 million trained weights scores 87.2, against 86.4 for full fine-tuning.

Hu et al. 2022, Table 2: RoBERTa-base, mean over the eight GLUE tasks. Full fine-tuning, BitFit and AdapterDrop (Rücklé et al.) as published in earlier work.

What happens when more weights are trained

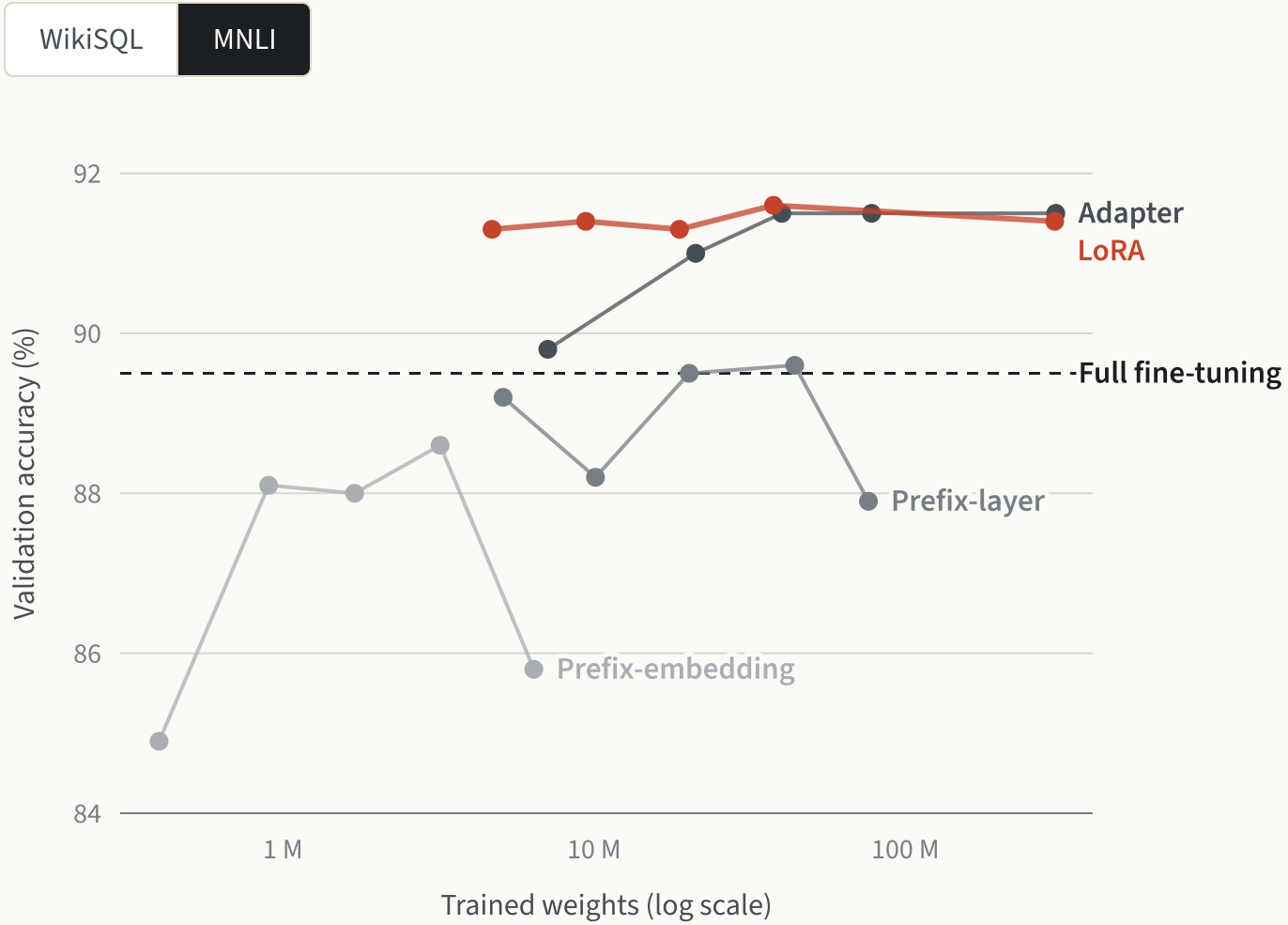


Each point is one size of a method. Prefix-embedding tuning improves up to 256 prompt tokens, then drops from 63.1 to 55.9. Prefix-layer tuning also falls after 20 million weights.

LoRA is flat from rank 1 to rank 64, at the level of full fine-tuning.

Hu et al. 2022, Table 15 (the points of Fig. 2): GPT-3 175B, validation accuracy; LoRA with equal rank on query and value, from rank 1 to 64. Full fine-tuning trains all 175 billion weights.

What happens when more weights are trained



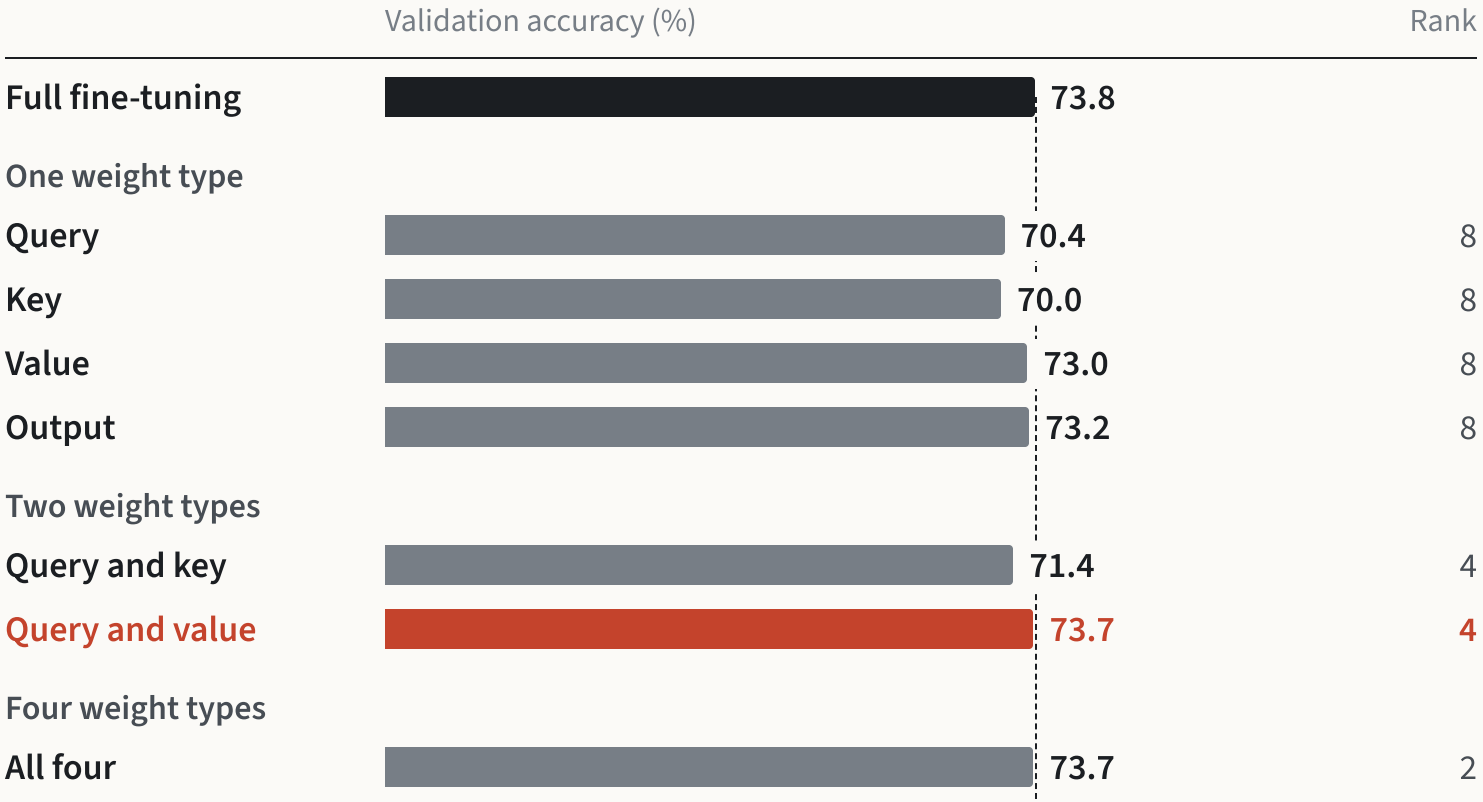
On MNLI the prompt methods again rise and fall, and the adapter needs about 40 million weights to catch up.

LoRA is above full fine-tuning at every size, so its rank is an easy choice.

Hu et al. 2022, Table 15 (the points of Fig. 2): GPT-3 175B, validation accuracy; LoRA with equal rank on query and value, from rank 1 to 64. Full fine-tuning trains all 175 billion weights.

Which weight matrices to adapt

WikiSQL | MNLI



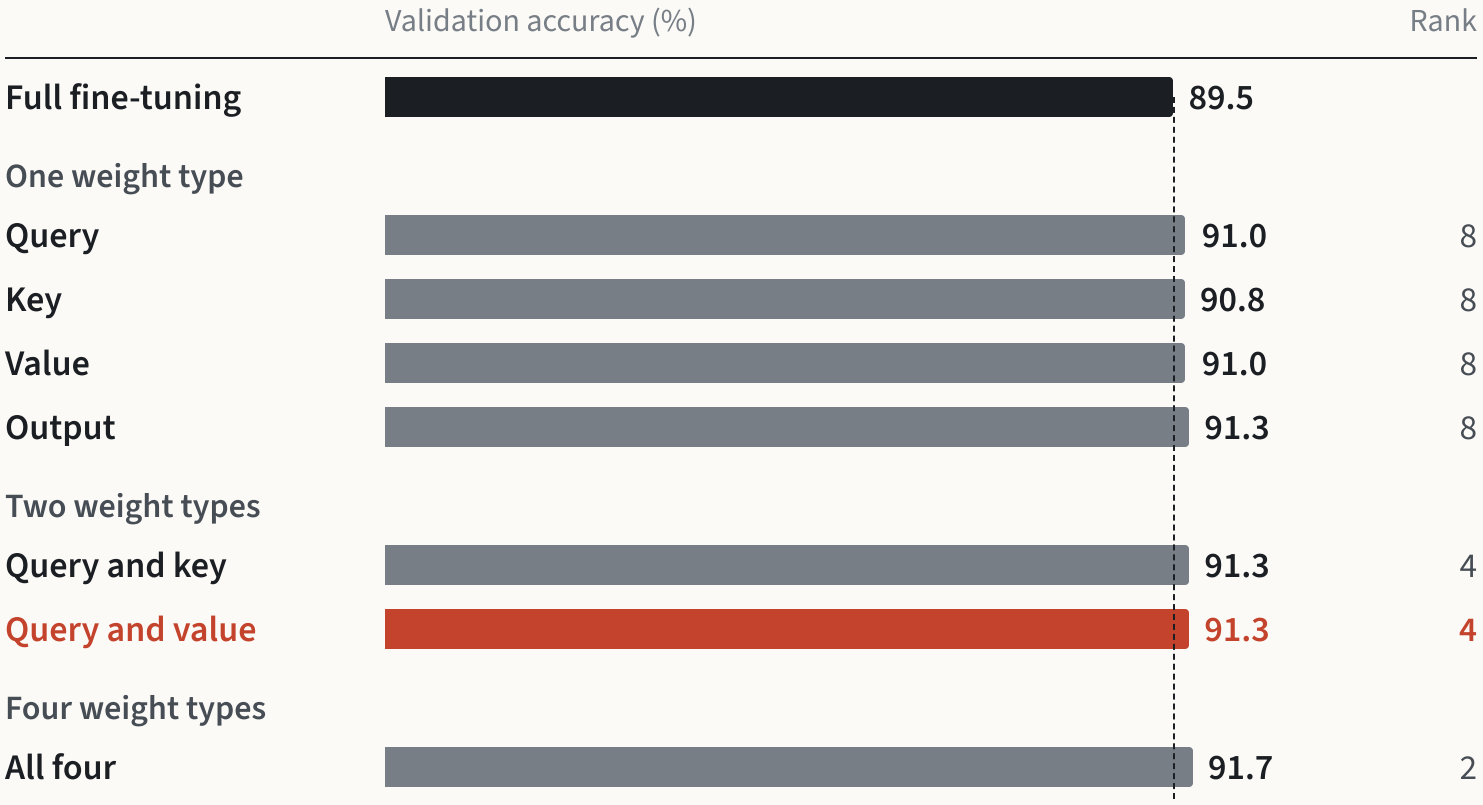
Every LoRA row trains the same 18 million weights. Putting them all into the query or the key weight is about three points worse than spreading them over query and value at rank 4.

Adapting all four at rank 2 does as well, so it pays to adapt more matrices with a smaller rank.

Hu et al. 2022, Table 5: GPT-3 175B with the same 18 million trained weights in every LoRA row, validation accuracy (noise ±0.5 on WikiSQL, ±0.1 on MNLI). Full fine-tuning: Table 4.

Which weight matrices to adapt

WikiSQL | **MNLI**

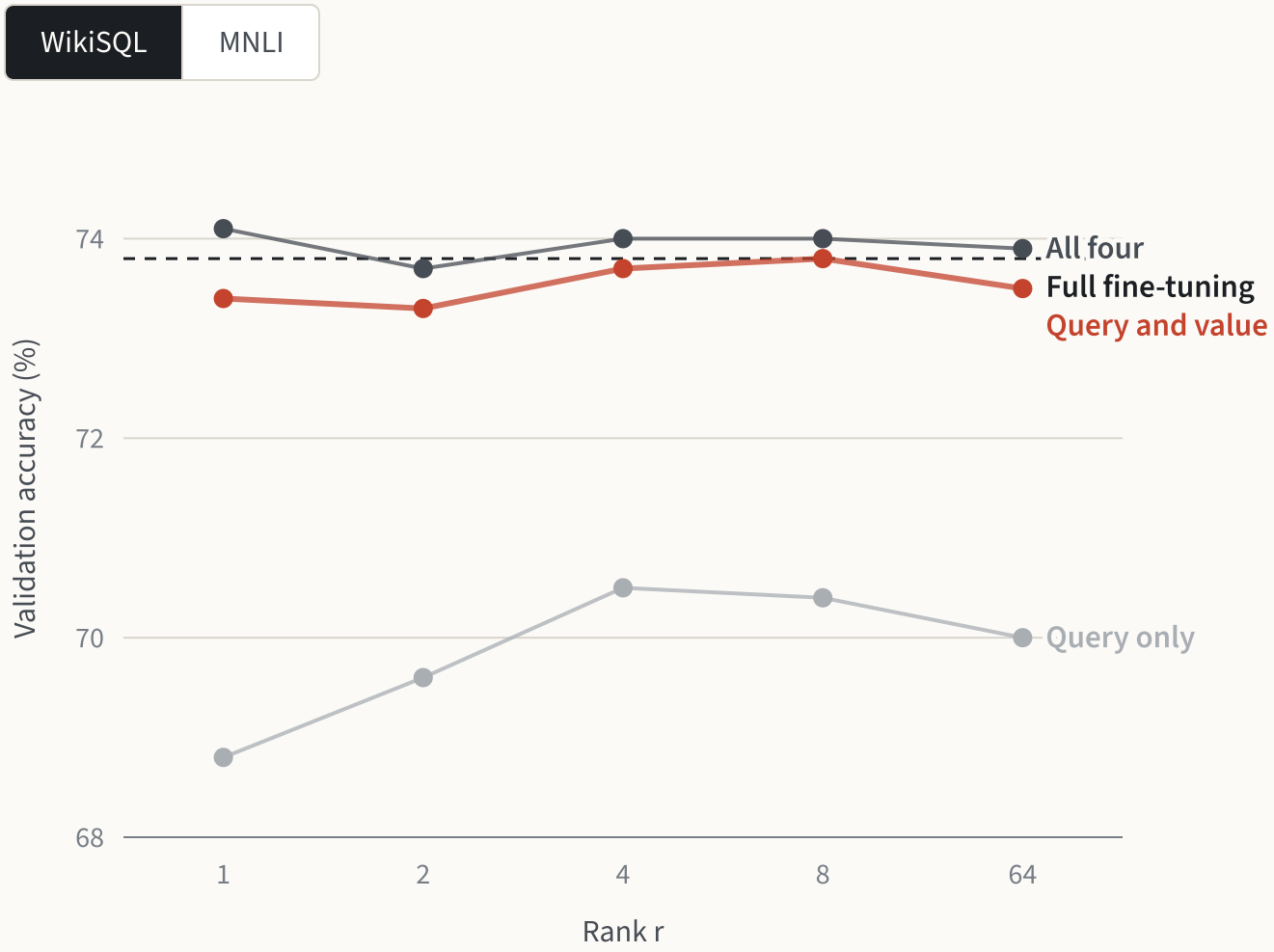


On MNLI the differences are smaller, but the order is similar: the query or key alone is lowest, and all four at rank 2 is highest, 91.7.

Every LoRA setting is above full fine-tuning here.

Hu et al. 2022, Table 5: GPT-3 175B with the same 18 million trained weights in every LoRA row, validation accuracy (noise ±0.5 on WikiSQL, ±0.1 on MNLI). Full fine-tuning: Table 4.

How small the rank can be

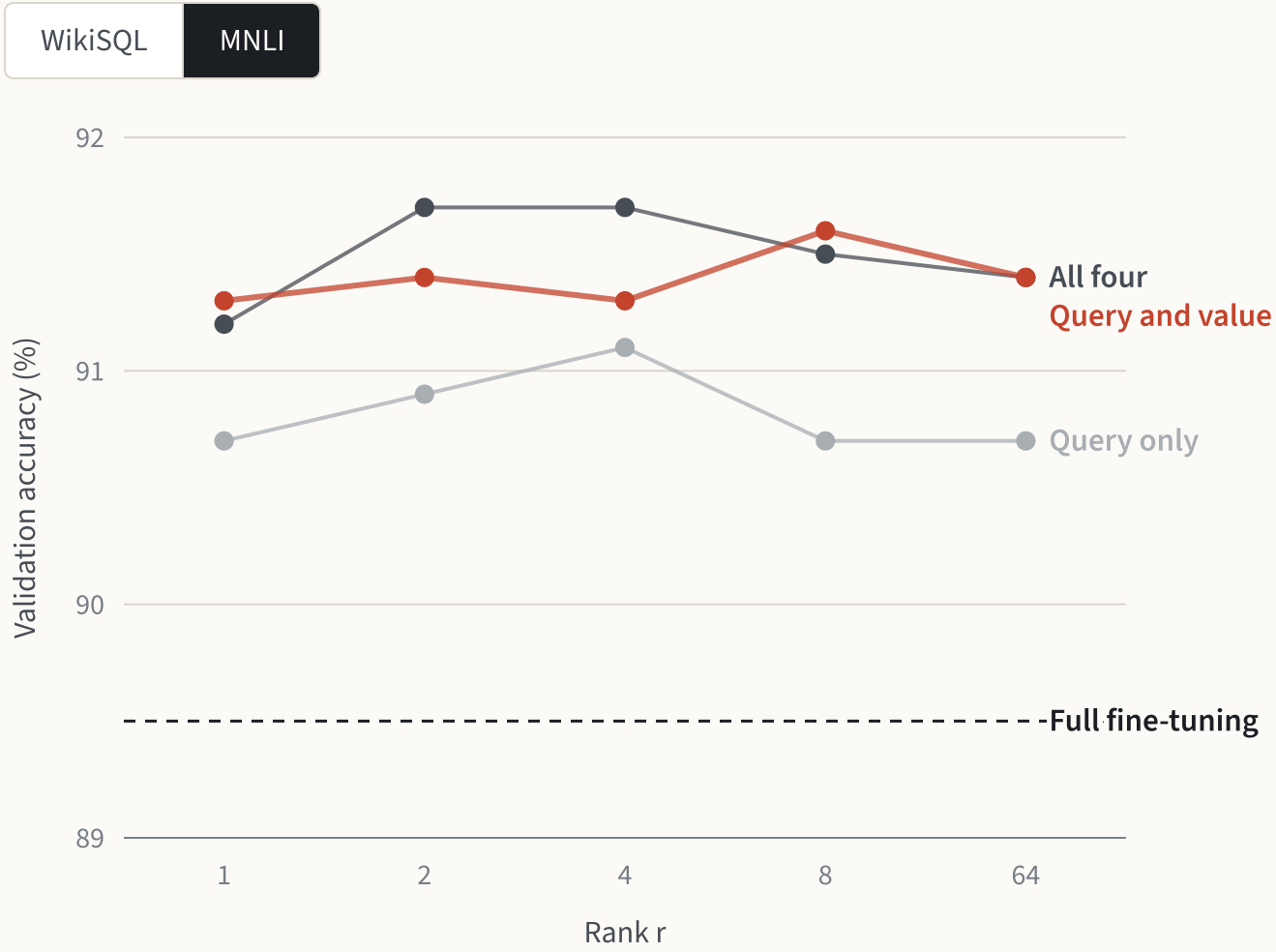


With the query and value weights together, rank 1 already gives 73.4, and no rank does better than 73.8, inside the noise of ±0.5.

The query weight alone needs rank 4 to reach its best, and stays three points lower.

Hu et al. 2022, Table 6: GPT-3 175B, validation accuracy at five ranks; noise ±0.5 on WikiSQL and ±0.1 on MNLI. On GPT-2 the best rank was between 4 and 16 (App. H.2).

How small the rank can be



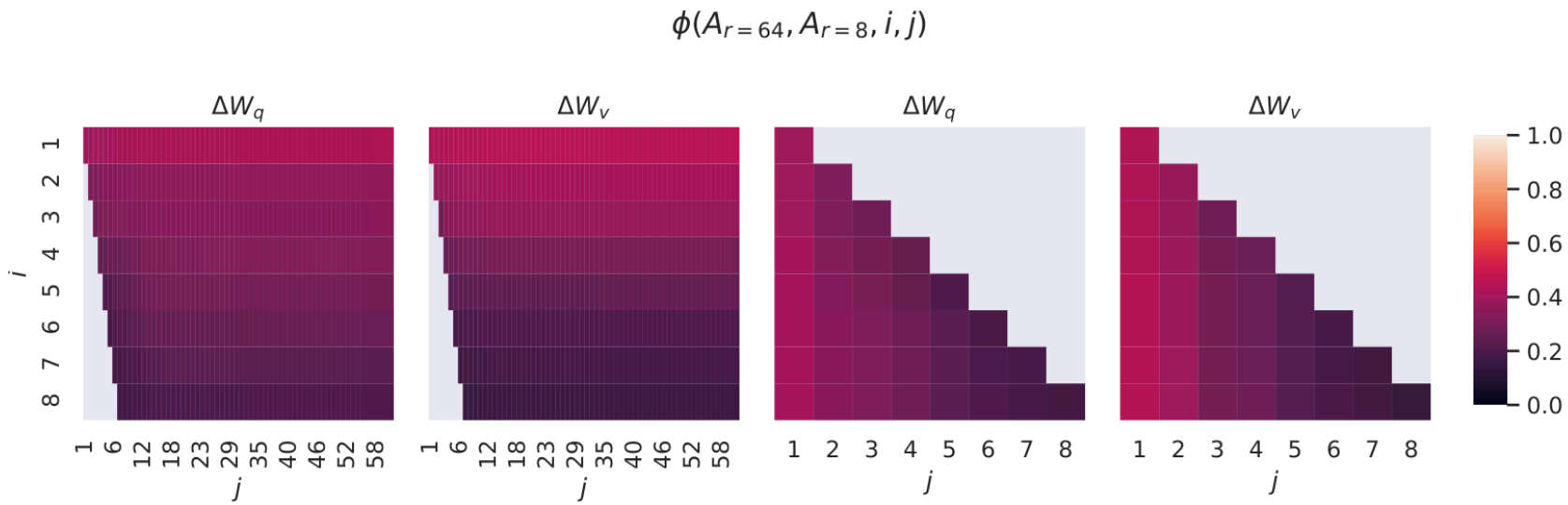
On MNLI, too, rank 1 for query and value is within 0.3 points of the best rank.

So the change fine-tuning needs has very few real directions, at least on GPT-3.

Hu et al. 2022, Table 6: GPT-3 175B, validation accuracy at five ranks; noise ±0.5 on WikiSQL and ±0.1 on MNLI. On GPT-2 the best rank was between 4 and 16 (App. H.2).

Why a small rank is enough

Rank 8 against rank 64 | The change against the weight



The authors trained rank 8 and rank 64 separately and compared the directions each learned. Bright means the two runs found the same direction. Only the first direction is clearly shared, with similarity above 0.5, and the rest overlap little, so most of the extra rank holds noise.

Hu et al. 2022, Fig. 3 (layer 48 of 96, GPT-3; values are normalized subspace similarity, 0 to 1) and Table 7 (rank 4, Frobenius norms).

Why a small rank is enough

Rank 8 against rank 64 | **The change against the weight**

21.67 is how strongly the pretrained weight W_q acts along its own top 4 directions.

0.32 is how strongly it acts along the 4 directions of the learned change ΔW_q: weak, but more than the 0.02 of random directions.

6.91 is the strength of the change along those directions, about 21 times what the pretrained weight had there.

> The change does not repeat what the model already does. It strongly amplifies directions the model learned in pretraining but used weakly.

Hu et al. 2022, Fig. 3 (layer 48 of 96, GPT-3; values are normalized subspace similarity, 0 to 1) and Table 7 (rank 4, Frobenius norms).

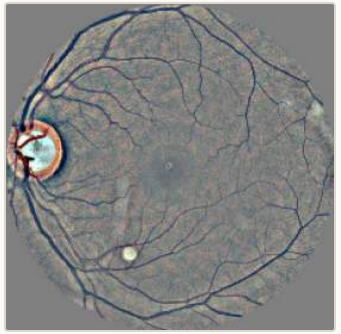
LoRA on images

The paper itself has no image experiments. Vision groups tested it next, first on the same 19 VTAB-1k tasks as VPT, then on medical images.

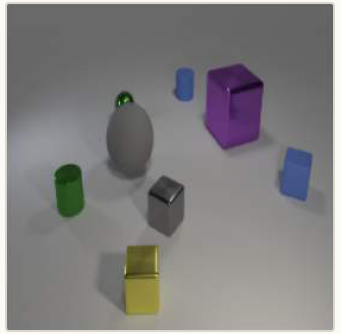
Both use a ViT-B/16, so the numbers sit next to last time's.



Natural



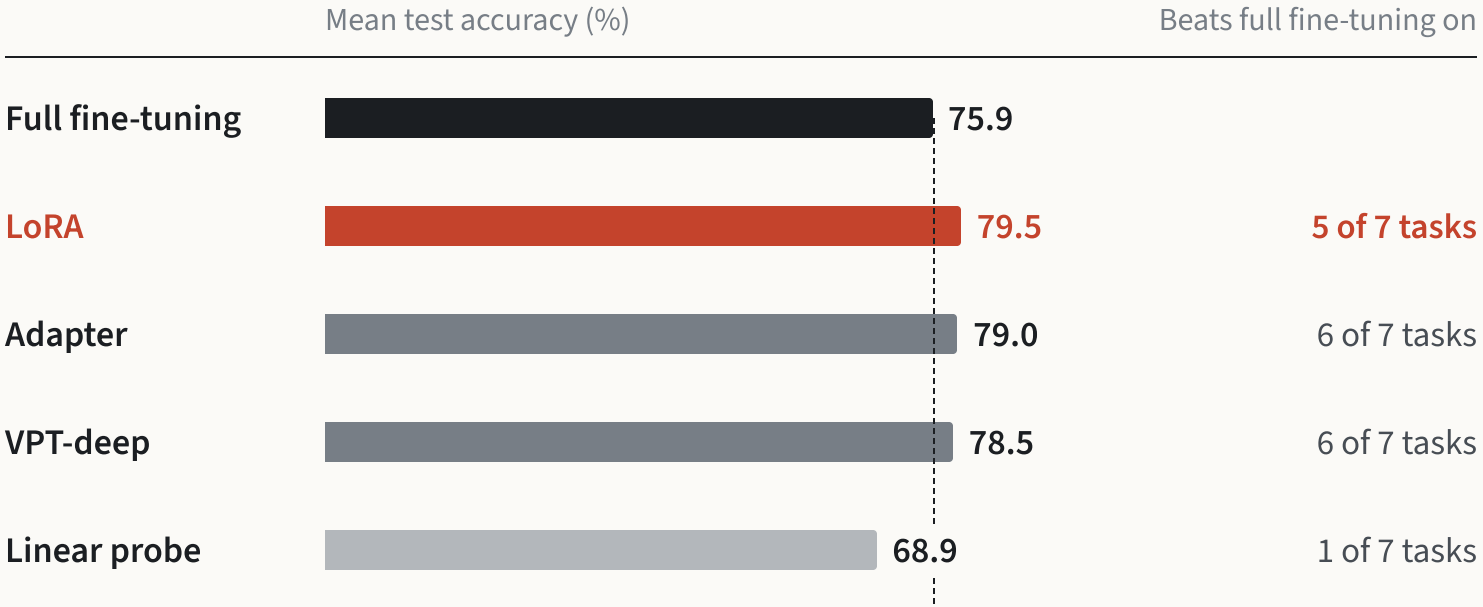
Specialized



Structured

LoRA, VPT and adapters on the same ViT

Natural | Specialized | Structured | All 19 tasks

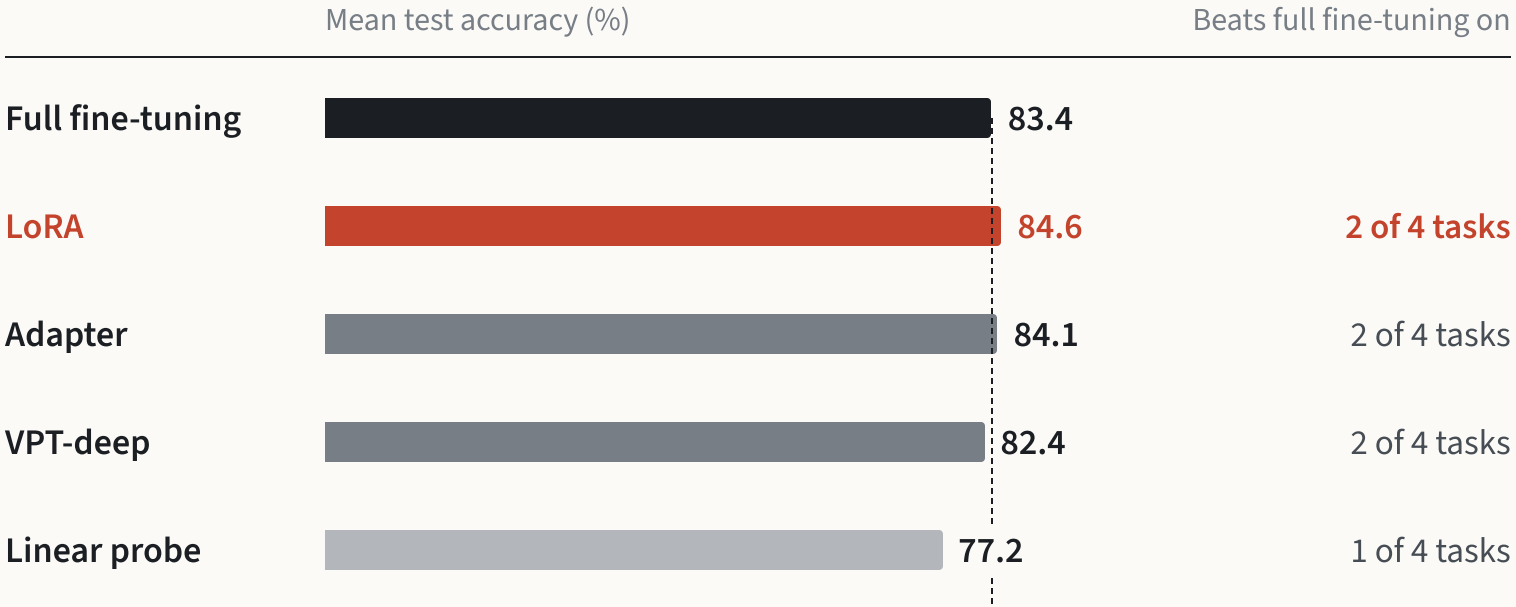


Ordinary photos, with 1,000 training images per task. LoRA, the adapter and VPT-deep all beat full fine-tuning by two to four points, and LoRA is narrowly the best.

Zhang et al. 2022, Table 1: ViT-B/16 pretrained on ImageNet-21k, VTAB-1k test accuracy. LoRA (rank 8) and the adapter were trained by Zhang et al.; the other rows are Jia et al. 2022, Table 1. Group means computed from the per-task values; "all 19" is the mean of the three groups.

LoRA, VPT and adapters on the same ViT

Natural | **Specialized** | Structured | All 19 tasks

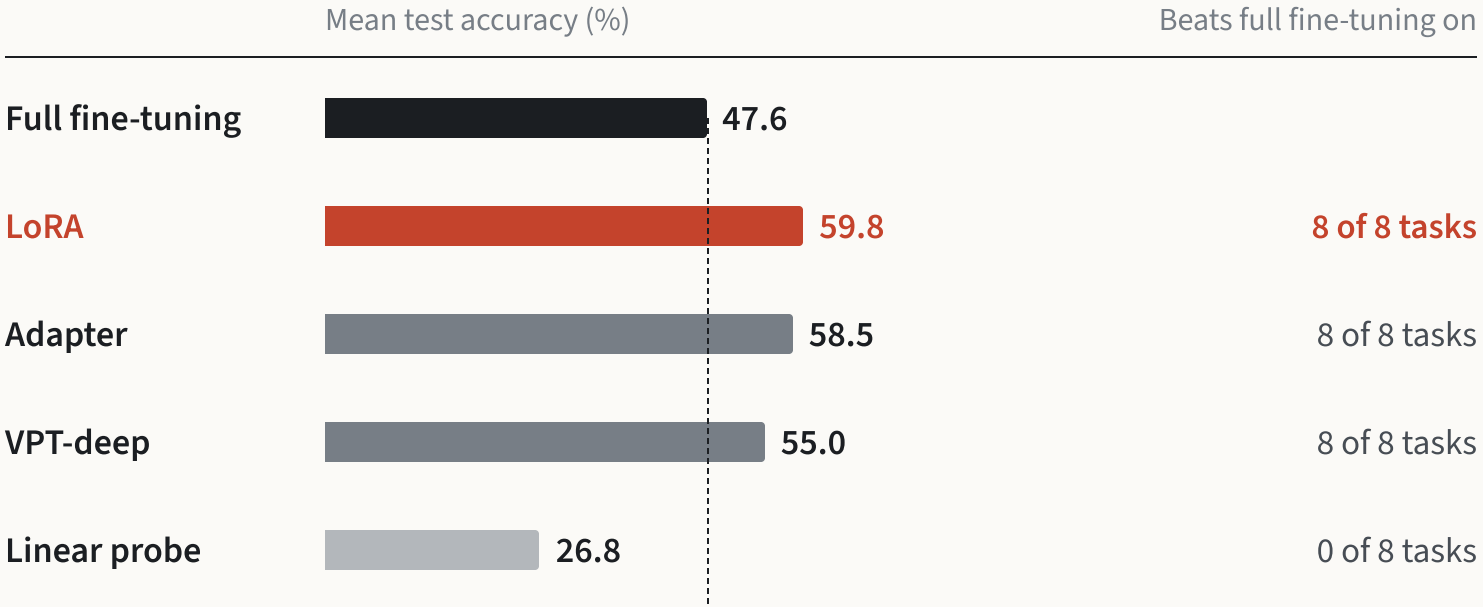


Satellite, microscope and retina images. LoRA and the adapter are slightly above full fine-tuning, 84.6 and 84.1 against 83.4, while VPT-deep is a point below.

Zhang et al. 2022, Table 1: ViT-B/16 pretrained on ImageNet-21k, VTAB-1k test accuracy. LoRA (rank 8) and the adapter were trained by Zhang et al.; the other rows are Jia et al. 2022, Table 1. Group means computed from the per-task values; "all 19" is the mean of the three groups.

LoRA, VPT and adapters on the same ViT

Natural | Specialized | **Structured** | All 19 tasks

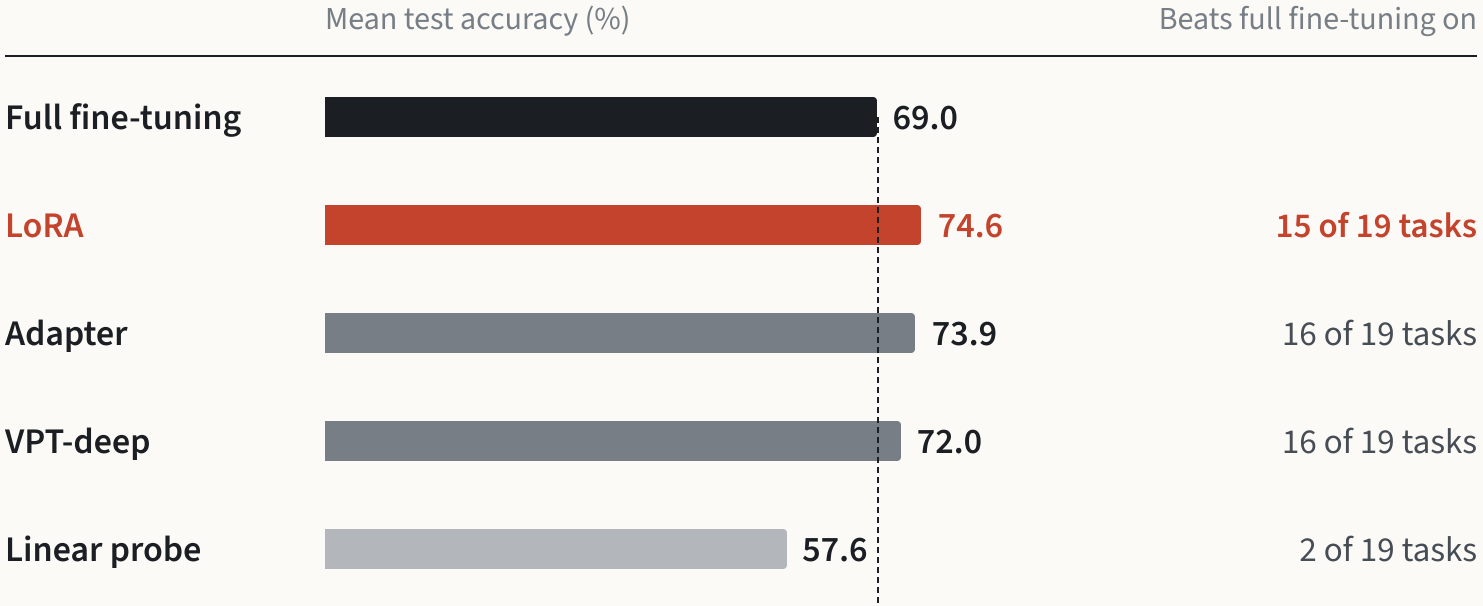


Counting, distances and angles, far from ImageNet. All three cheap methods beat full fine-tuning on all eight tasks, LoRA by the most: 59.8 against 47.6, with VPT-deep at 55.0.

Zhang et al. 2022, Table 1: ViT-B/16 pretrained on ImageNet-21k, VTAB-1k test accuracy. LoRA (rank 8) and the adapter were trained by Zhang et al.; the other rows are Jia et al. 2022, Table 1. Group means computed from the per-task values; "all 19" is the mean of the three groups.

LoRA, VPT and adapters on the same ViT

Natural | Specialized | Structured | **All 19 tasks**

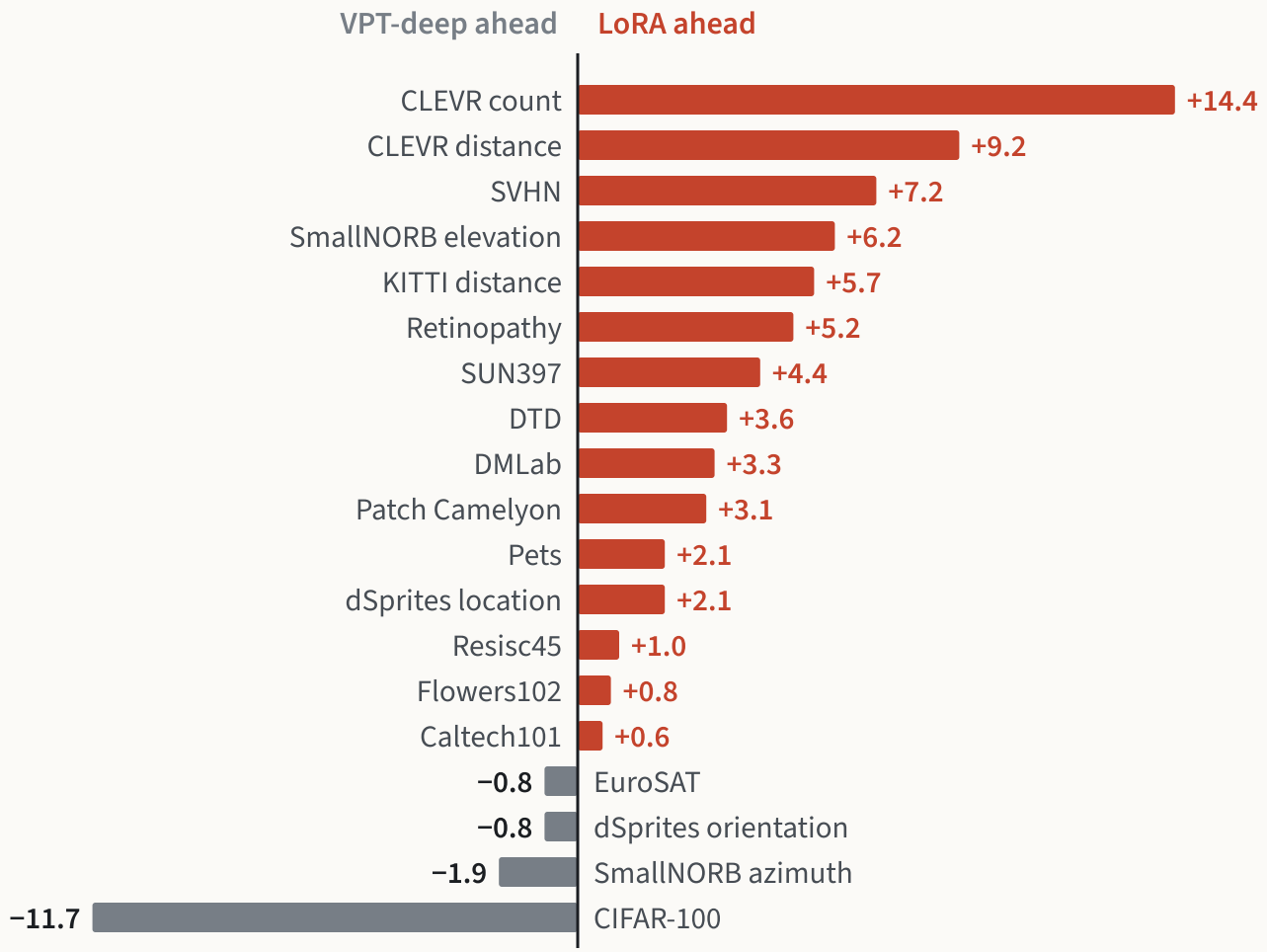


Over all three groups LoRA reaches 74.6, VPT-deep 72.0 and full fine-tuning 69.0.

LoRA trains 0.29 million weights here and VPT-deep 0.64 million on average, so LoRA is both smaller and more accurate.

Zhang et al. 2022, Table 1: ViT-B/16 pretrained on ImageNet-21k, VTAB-1k test accuracy. LoRA (rank 8) and the adapter were trained by Zhang et al.; the other rows are Jia et al. 2022, Table 1. Group means computed from the per-task values; "all 19" is the mean of the three groups.

Neither LoRA nor VPT wins every task



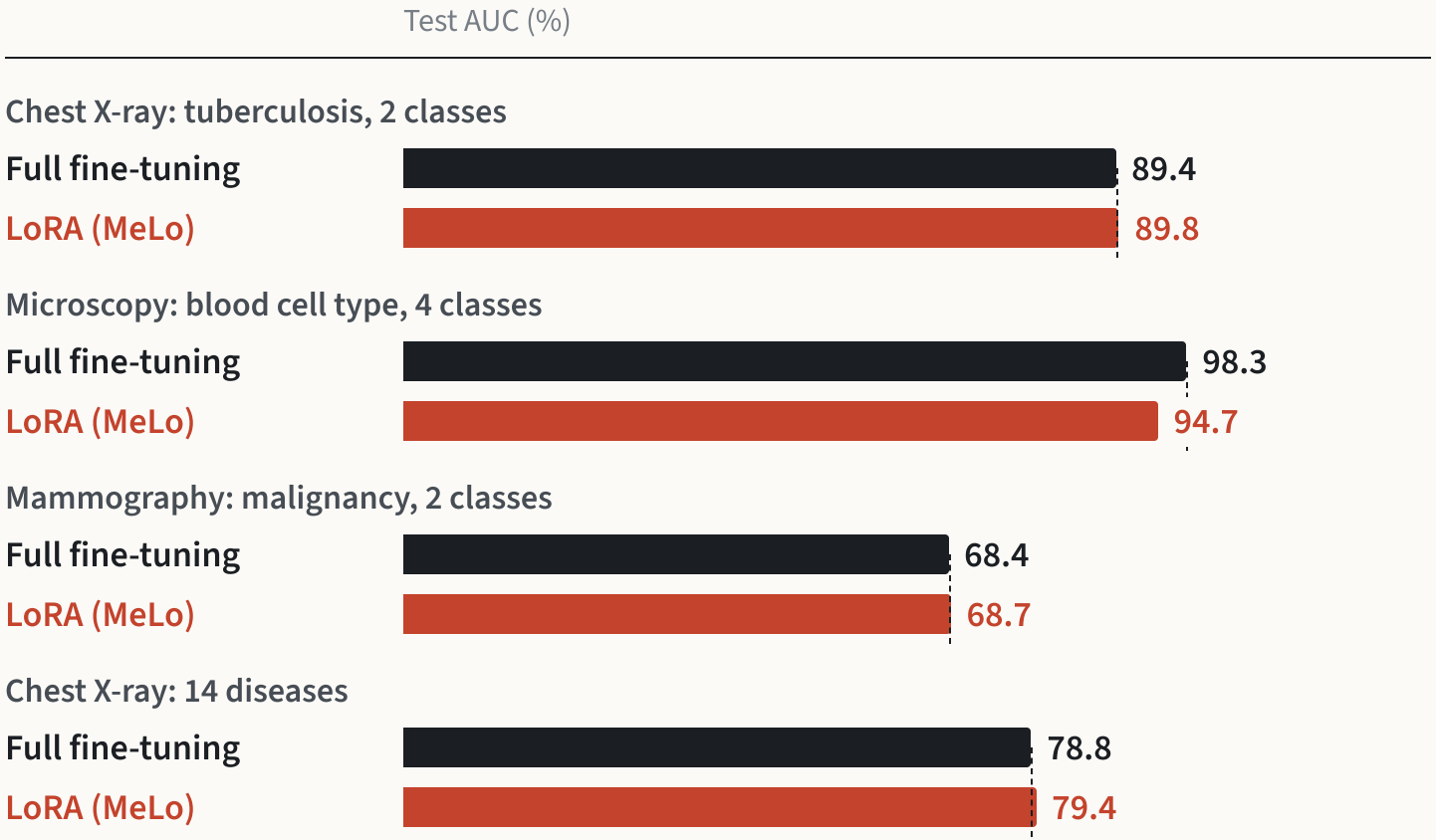
LoRA is ahead on 15 of the 19 tasks, by up to 14 points on counting objects in CLEVR. VPT-deep is ahead on 4, and on CIFAR-100 by 12 points.

Zhang et al. conclude that no single method fits every task. Their method, NOAH, searches for a mix of LoRA, adapter and prompts per task, and reaches 75.5 on average.

Zhang et al. 2022, Table 1: per-task test accuracy of LoRA minus VPT-deep on VTAB-1k, ViT-B/16. Right of the line, LoRA is ahead.

LoRA on medical images

AUC | Accuracy | Switching tasks



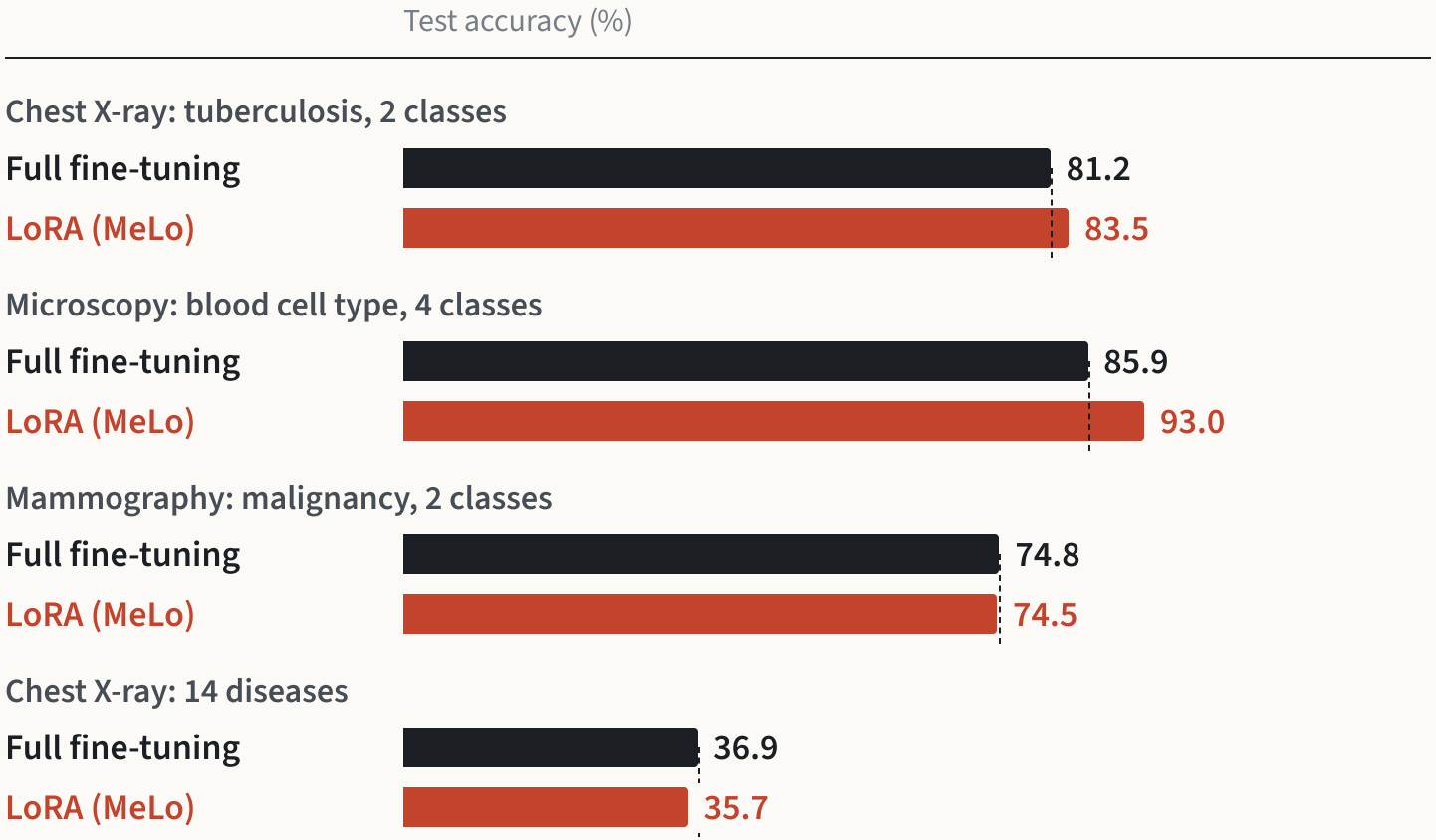
MeLo adds LoRA pairs of rank 4 to the query and value weights of an ImageNet ViT-B, training 0.14 million weights, 0.17% of the model.

Its AUC is within a point of full fine-tuning on three tasks and 3.6 points lower on blood cells.

Zhu et al. 2024 (MeLo, ISBI), Table 1: ViT-B pretrained on ImageNet, LoRA rank 4 on query and value, single runs on their own test splits. The 14-disease task is multi-label.

LoRA on medical images

AUC | **Accuracy** | Switching tasks



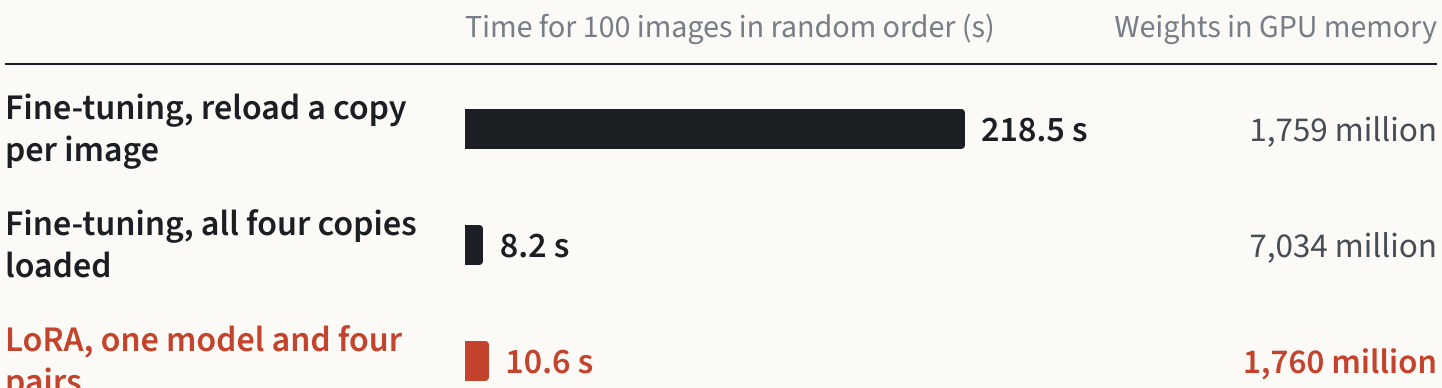
In accuracy MeLo is ahead on two tasks and slightly behind on two.

These are single runs on small test sets, about 80 mammograms for INbreast, so the fair reading of the title, "better than fine-tuning", is "about the same".

Zhu et al. 2024 (MeLo, ISBI), Table 1: ViT-B pretrained on ImageNet, LoRA rank 4 on query and value, single runs on their own test splits. The 14-disease task is multi-label.

LoRA on medical images

AUC | Accuracy | **Switching tasks**



Their practical point is switching between tasks, measured with a giant CLIP ViT on 25 images from each task in random order.

Reloading a fine-tuned copy for each image takes 218 s, and keeping all four copies loaded needs four times the memory. One frozen model with four LoRA pairs takes 10.6 s at the memory of one model.

Zhu et al. 2024, Table 2: ViT-giga pretrained with CLIP (1.76 billion weights), 100 images, 25 from each of the four datasets, in random order, on one A100 GPU.

Where LoRA falls short

The rank and the weights are chosen by hand

The paper picks the query and value weights and a small rank by trying settings, and leaves a principled choice to future work. On VTAB-1k the best method even changes from task to task.

Merged weights serve one task at a time

Once BA is added into W, every input in a batch must belong to that task. Mixing tasks in one batch means keeping the pairs separate, which brings back a small extra cost; VPT's prompts mix freely.

A small rank assumes a nearby task

The authors expect a small rank to fail when the new task is far from pretraining, such as a different language. The low-rank analysis was only done on GPT-3, not on vision models.

Training still runs through the whole model

LoRA saves the optimizer state of the frozen weights, but the gradient still passes back through every layer. On ViT-B most memory is activations: freezing the backbone for VPT only cut it from 11.7 to 10.3 GB.

Hu et al. 2022, Sec. 4.2 (batching), footnote 6 (a different language), Sec. 7 and 8 (analysis on GPT-3, future work). Zhang et al. 2022, Fig. 1b. Jia et al. 2022, Table 12 (ViT-B/16, batch 64).

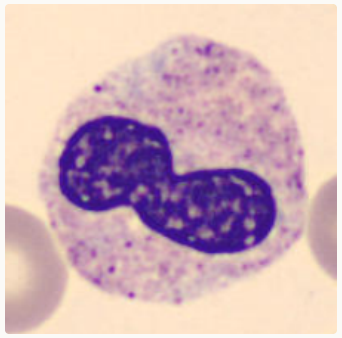
My experiments

I trained LoRA on the same three medical datasets as my VPT runs, with the same base model, data and training recipe.

So it sits next to VPT-deep, a linear probe and full fine-tuning. First the results, then the models running live.



Skin lesions



Blood cells



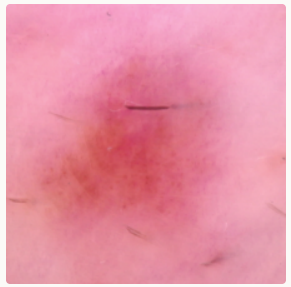
Chest X-rays

I tested LoRA on the same three medical datasets

Each method was trained twice: on 1,000 images, as in VTAB-1k, and on the full training set.



melanoma

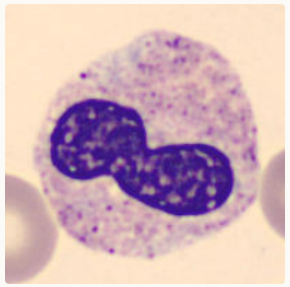


nevus

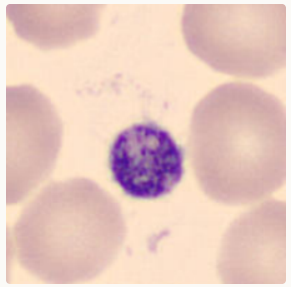
Skin lesions

Dermoscopy, 7 classes

1,000 or all **7,007** training images



neutrophil



platelet

Blood cells

Microscopy, 8 cell types

1,000 or all **11,959** training images



normal



pneumonia

Chest X-rays

Radiographs, pneumonia or normal

1,000 or all **4,708** training images

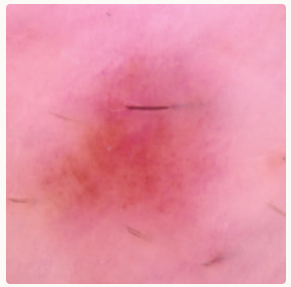
MedMNIST+ at 224 × 224 (Yang et al. 2023); skin lesions from HAM10000 (Tschandl et al. 2018). RTX 5070 Ti laptop GPU. SGD for linear probe and VPT, AdamW for LoRA and full fine-tuning; learning rate picked on validation.

I tested LoRA on the same three medical datasets

Each method was trained twice: on 1,000 images, as in VTAB-1k, and on the full training set.



melanoma

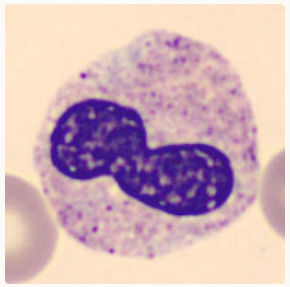


nevus

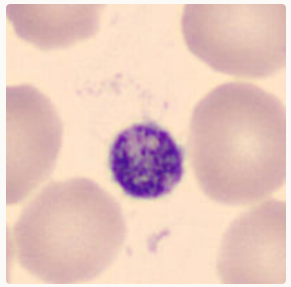
Skin lesions

Dermoscopy, 7 classes

1,000 or all **7,007** training images



neutrophil



platelet

Blood cells

Microscopy, 8 cell types

1,000 or all **11,959** training images



normal

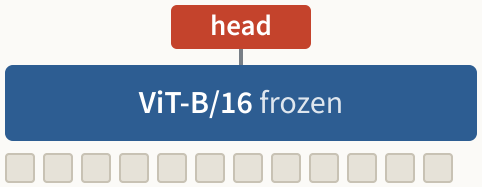


pneumonia

Chest X-rays

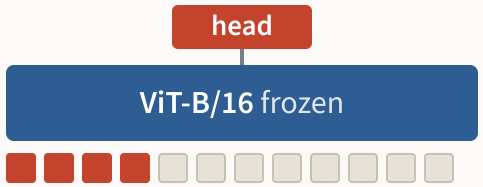
Radiographs, pneumonia or normal

1,000 or all **4,708** training images



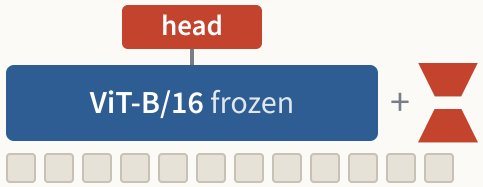
Linear probe

Trains only a new head.



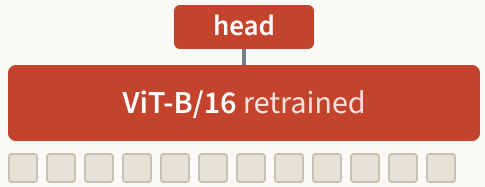
VPT-deep

Trains a head and 50 prompts per layer.



LoRA

Trains a head and rank-8 pairs beside W_q and W_v.



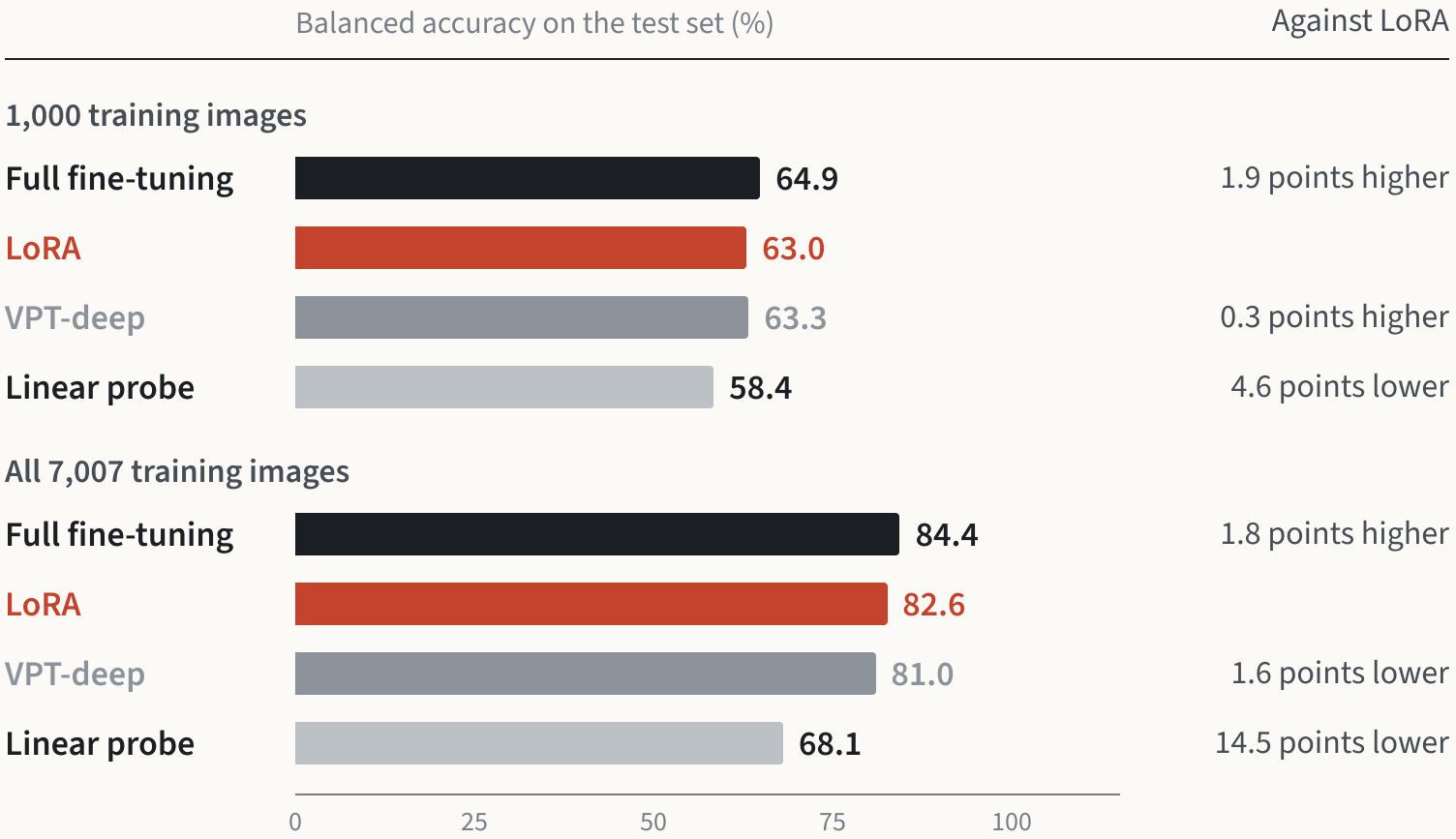
Full fine-tuning

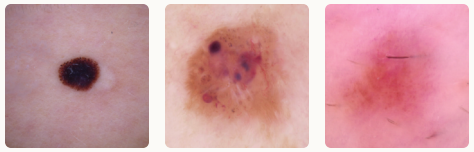
Retrains all 85.8 million weights.

MedMNIST+ at 224 × 224 (Yang et al. 2023); skin lesions from HAM10000 (Tschandl et al. 2018). RTX 5070 Ti laptop GPU. SGD for linear probe and VPT, AdamW for LoRA and full fine-tuning; learning rate picked on validation.

How LoRA compares on my three datasets

Skin lesions | Blood cells | Chest X-rays





Seven kinds of skin lesion, and two thirds of the images show harmless moles. This is the hardest of the three tasks.

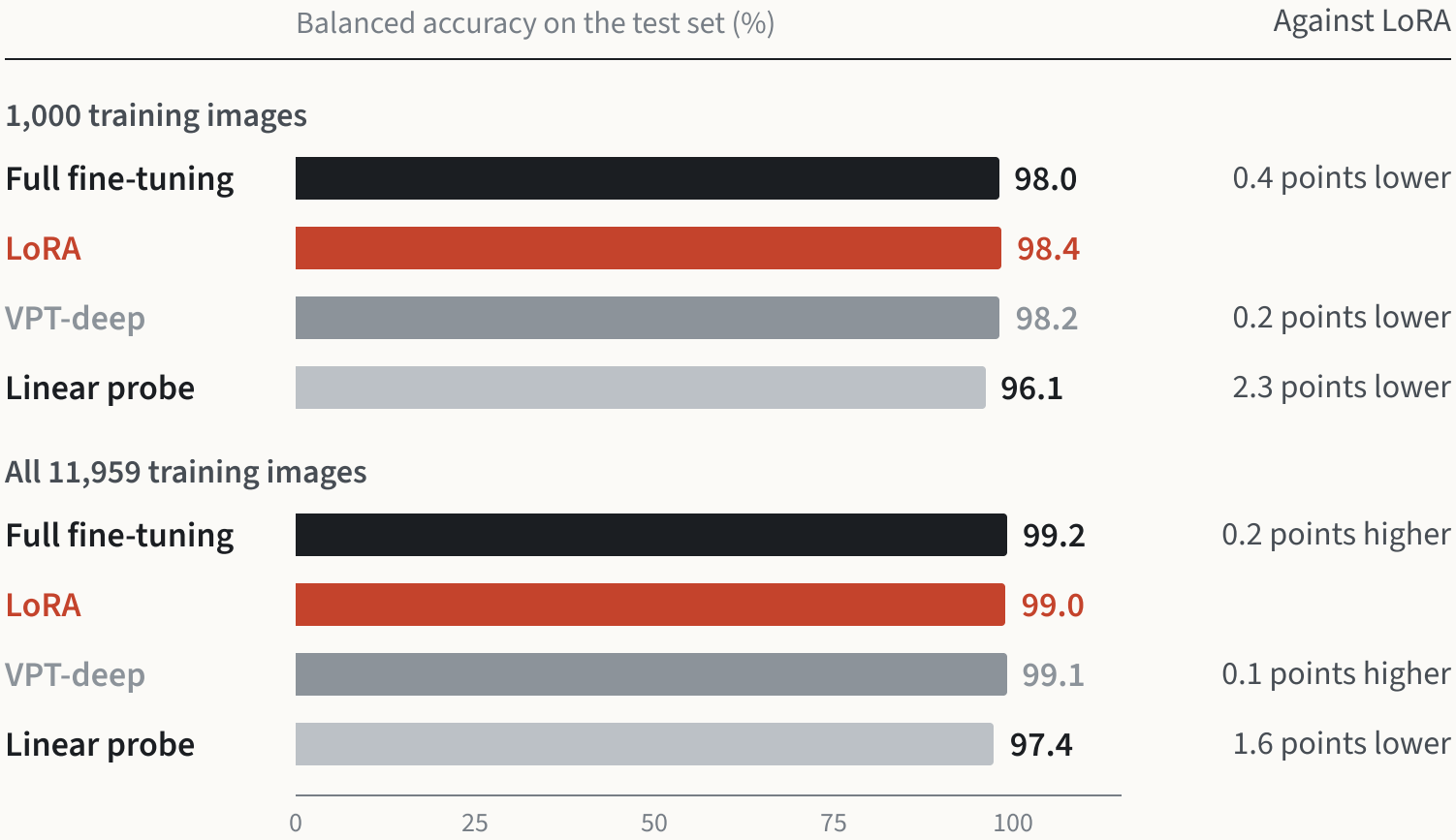
With all 7,007 images, LoRA is 1.6 points ahead of VPT-deep and 1.8 points behind full fine-tuning.

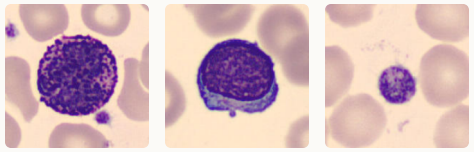
With 1,000 images, LoRA is 0.3 points behind VPT-deep and 1.9 points behind full fine-tuning.

My runs: ViT-B/16 (ImageNet-21k), MedMNIST+ 224 × 224, balanced accuracy on the test set (the mean per-class recall), bars start at 0. Each method was trained once with 1,000 images and once with the full training set; there are no points in between.

How LoRA compares on my three datasets

Skin lesions | **Blood cells** | Chest X-rays





Eight kinds of blood cell under a microscope. Every method does well here, so the bars are almost the same length.

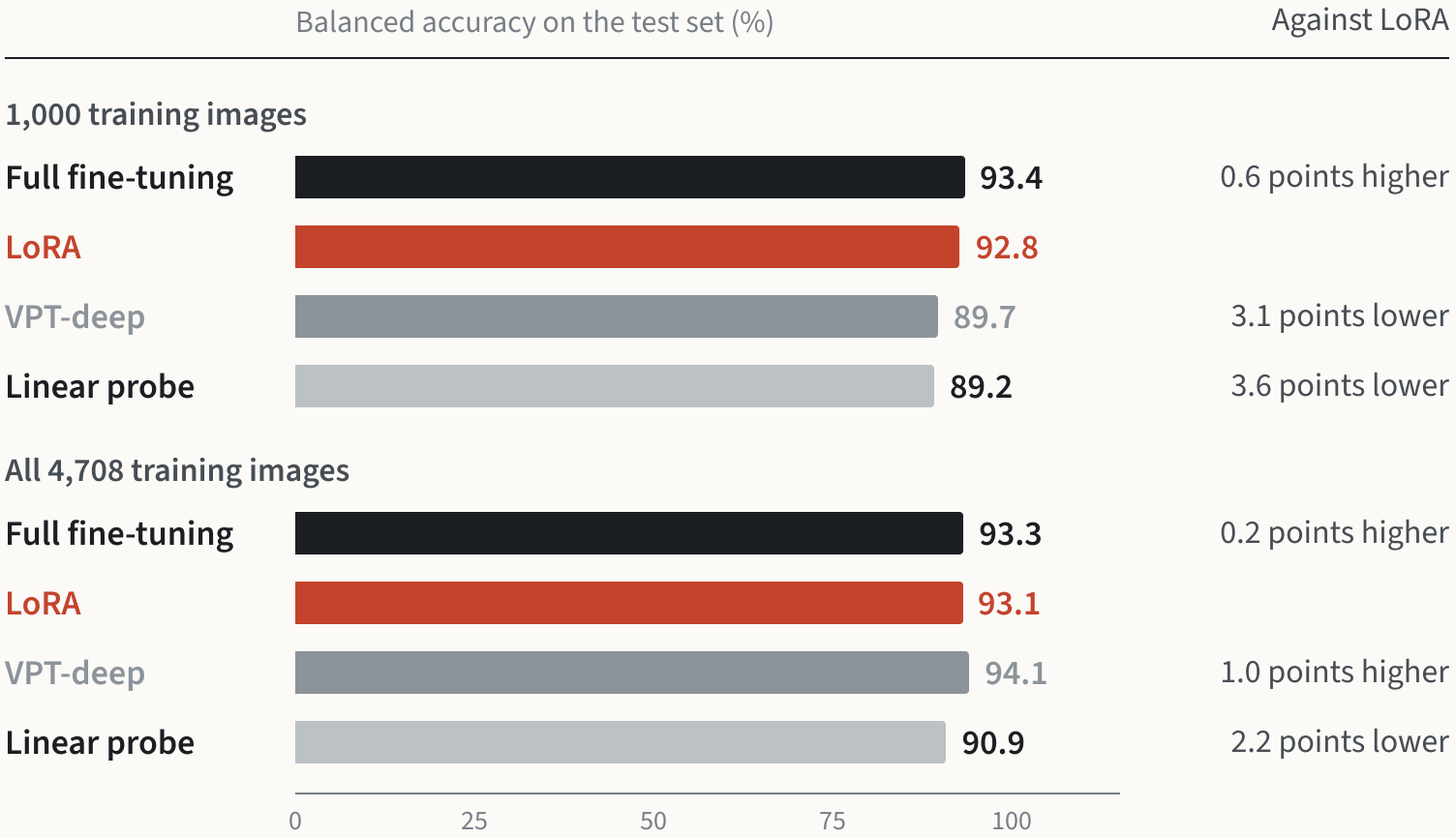
With all 11,959 images, LoRA is 0.1 points behind VPT-deep and 0.2 points behind full fine-tuning.

With 1,000 images, LoRA is 0.2 points ahead of VPT-deep and 0.4 points ahead of full fine-tuning.

My runs: ViT-B/16 (ImageNet-21k), MedMNIST+ 224 × 224, balanced accuracy on the test set (the mean per-class recall), bars start at 0. Each method was trained once with 1,000 images and once with the full training set; there are no points in between.

How LoRA compares on my three datasets

Skin lesions | Blood cells | **Chest X-rays**





Chest X-rays of children, labelled normal or pneumonia.

With all 4,708 images, LoRA is 1.0 points behind VPT-deep and 0.2 points behind full fine-tuning.

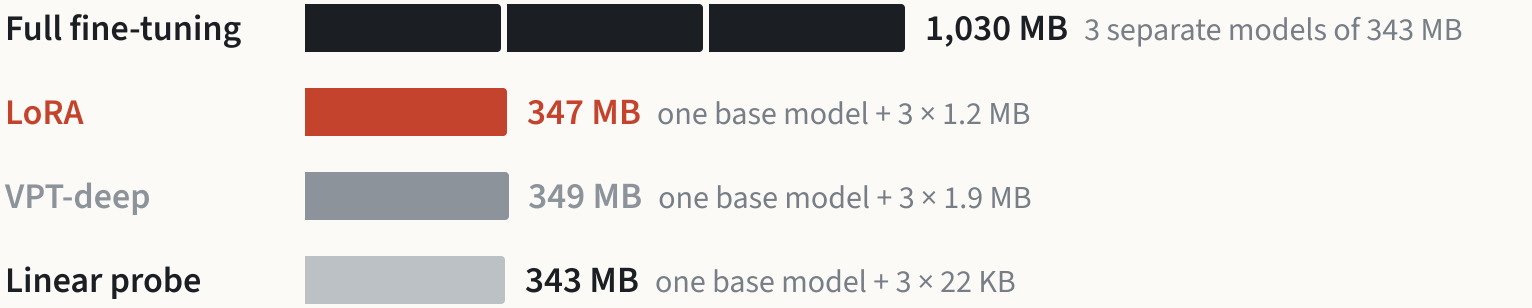
With 1,000 images, LoRA is 3.1 points ahead of VPT-deep and 0.6 points behind full fine-tuning.

My runs: ViT-B/16 (ImageNet-21k), MedMNIST+ 224 × 224, balanced accuracy on the test set (the mean per-class recall), bars start at 0. Each method was trained once with 1,000 images and once with the full training set; there are no points in between.

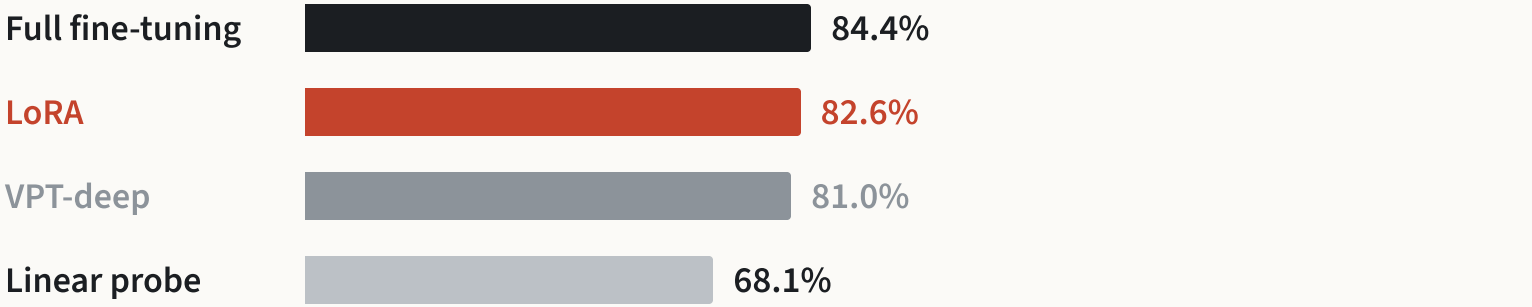
What each method costs on skin lesions

Storage and accuracy | Training | Inference

Stored for all three tasks



Balanced accuracy on the test set



LoRA stores 1.2 MB per task and VPT-deep 1.9 MB, against 343 MB for a fine-tuned copy. Both share one frozen base model across all tasks.

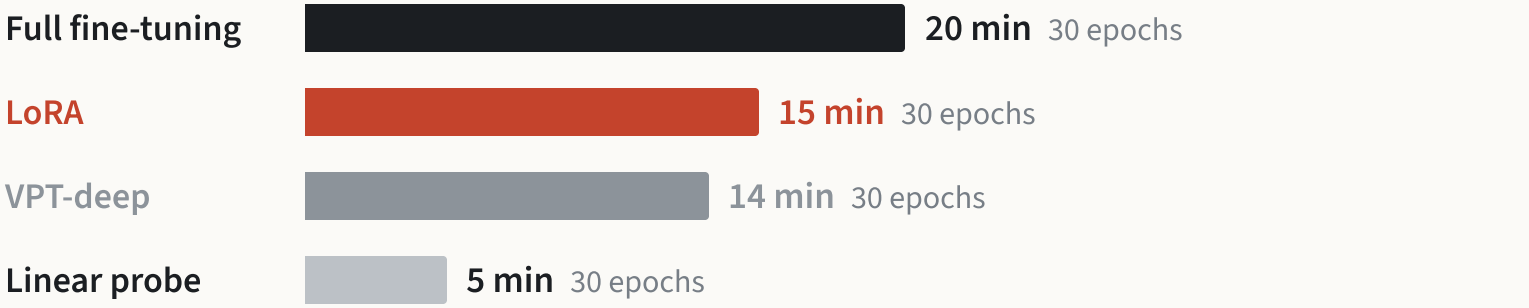
Here LoRA reaches 82.6% and VPT-deep 81.0%, against 84.4% for full fine-tuning and 68.1% for the linear probe.

Skin lesions (DermaMNIST), all 7,007 training images. RTX 5070 Ti Laptop GPU, batch 32; CPU inference on 2 threads; 32-bit weights; LoRA rank 8 on the query and value weights.

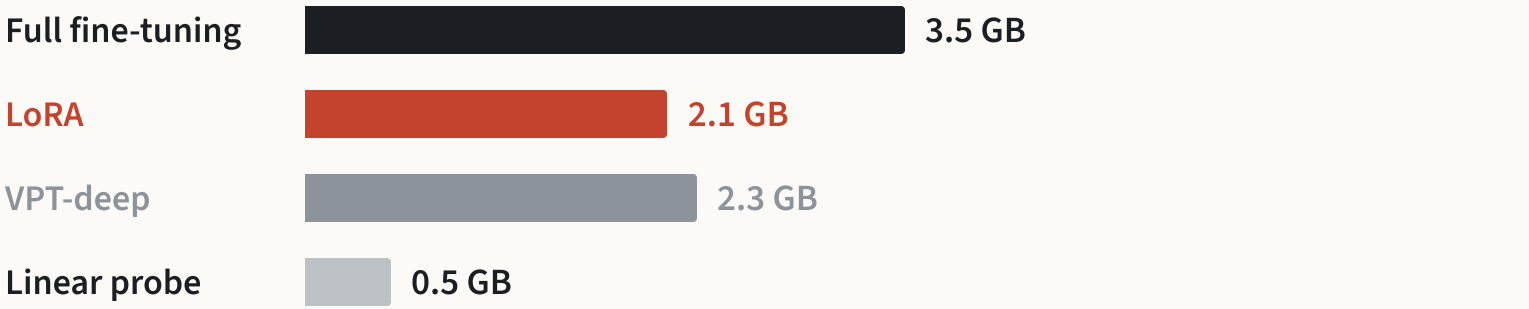
What each method costs on skin lesions

Storage and accuracy | **Training** | Inference

Training time on all 7,007 images



Peak GPU memory during training, batch 32



LoRA needed 2.1 GB of GPU memory and VPT-deep 2.3 GB, against 3.5 GB for full fine-tuning.

Both still pass the error signal back through all twelve frozen layers, so only the linear probe is really cheap to train.

Skin lesions (DermaMNIST), all 7,007 training images. RTX 5070 Ti Laptop GPU, batch 32; CPU inference on 2 threads; 32-bit weights; LoRA rank 8 on the query and value weights.

What each method costs on skin lesions

Storage and accuracy | Training | **Inference**

Time per image on the CPU, 2 threads



Tokens that every layer processes



VPT-deep needs 250 ms per image, because every layer processes 50 extra tokens. LoRA with separate pairs needs 190 ms.

Merged into the weights, LoRA takes 188 ms, the time of the unchanged model (189 ms).

Skin lesions (DermaMNIST), all 7,007 training images. RTX 5070 Ti Laptop GPU, batch 32; CPU inference on 2 threads; 32-bit weights; LoRA rank 8 on the query and value weights.

Live demo: one image, three models

Skin lesion, one test image, models trained on **all training images**.



True label: **melanoma**

Linear probe

shared frozen model + head

68.5% test accuracy

melanocytic nevi	**73.7%**
melanoma	**23.8%**
benign keratosis-like	**2.1%**

Wrong

177 ms per image

LoRA

shared frozen model + pairs

82.5% test accuracy

melanocytic nevi	**92.8%**
melanoma	**7.1%**
benign keratosis-like	**0.0%**

Wrong

172 ms per image

Full fine-tuning

its own retrained model

84.4% test accuracy

melanoma	**97.7%**
melanocytic nevi	**2.2%**
benign keratosis-like	**0.0%**

Correct

157 ms per image

Test accuracy is balanced accuracy on the whole test set, for exactly these models. Changing the training set swaps one 1.2 MB file for LoRA, but a whole second 172 MB model for full fine-tuning.

A recorded run. In the HTML version of this deck the three models run live in the browser.

Live demo: one image, three models

Skin lesion, one test image, models trained on **1,000 training images**.



True label: **melanoma**

Linear probe

shared frozen model + head

58.1% test accuracy

melanocytic nevi	**84.5%**
benign keratosis-like	**7.9%**
melanoma	**7.5%**

Wrong

173 ms per image

LoRA

shared frozen model + pairs

63.0% test accuracy

melanocytic nevi	**86.6%**
melanoma	**12.8%**
benign keratosis-like	**0.4%**

Wrong

149 ms per image

Full fine-tuning

its own retrained model

64.8% test accuracy

melanocytic nevi	**95.1%**
melanoma	**4.2%**
benign keratosis-like	**0.2%**

Wrong

184 ms per image

Test accuracy is balanced accuracy on the whole test set, for exactly these models. Changing the training set swaps one 1.2 MB file for LoRA, but a whole second 172 MB model for full fine-tuning.

A recorded run. In the HTML version of this deck the three models run live in the browser.

Summary

0.44%	of ViT-B's weights are trained with rank 8 on the query and value weights and a head: a 1.5 MB file per task.
1.00×	the test-time cost of the original model, because the learned change is added into the weights after training.
74.6	mean VTAB-1k accuracy on ViT-B/16, against 72.0 for VPT-deep and 69.0 for full fine-tuning.
r = 1	was already enough on GPT-3 for the query and value weights: the change fine-tuning needs is low-rank.
88.1%	average balanced accuracy of LoRA on my three medical datasets, against 87.6% for VPT-deep, 88.8% for full fine-tuning and 83.3% for the linear probe.

ViT-B/16 with a 102-class head, 32-bit numbers. Zhang et al. 2022, Table 1, mean of the three VTAB-1k groups; VPT-deep and full fine-tuning from Jia et al. 2022. Hu et al. 2022, Table 6. Medical datasets: my runs, balanced accuracy on the test set.

References

This paper

Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang & Chen 2022. *LoRA: low-rank adaptation of large language models.* ICLR. arXiv:2106.09685.

LoRA on images

Zhang, Zhou & Liu 2022. *Neural prompt search* (NOAH). arXiv:2206.04673.

Zhu et al. 2024. *MeLo: low-rank adaptation is better than fine-tuning for medical image diagnosis.* ISBI. arXiv:2311.08236.

Ideas LoRA builds on

Li, Farkhoor, Liu & Yosinski 2018. *Measuring the intrinsic dimension of objective landscapes.* ICLR.

Aghajanyan, Zettlemoyer & Gupta 2021. *Intrinsic dimensionality explains the effectiveness of language model fine-tuning.* ACL.

Methods compared

Houlsby et al. 2019. *Parameter-efficient transfer learning for NLP* (adapter). ICML.

Lin, Madotto & Fung 2020. *Exploring versatile generative language model via parameter-efficient transfer learning.* Findings of EMNLP.

Rücklé et al. 2021. *AdapterDrop: on the efficiency of adapters in Transformers.* EMNLP.

Li & Liang 2021. *Prefix-tuning: optimizing continuous prompts for generation.* ACL.

Ben Zaken, Goldberg & Ravfogel 2022. *BitFit.* ACL.

Jia et al. 2022. *Visual prompt tuning.* ECCV.

Models

Dosovitskiy et al. 2021. *An image is worth 16×16 words: Transformers for image recognition at scale* (ViT). ICLR.

Brown et al. 2020. *Language models are few-shot learners* (GPT-3). NeurIPS.

Radford et al. 2019. *Language models are unsupervised multitask learners* (GPT-2). OpenAI report.

Liu et al. 2019. *RoBERTa: a robustly optimized BERT pretraining approach.* arXiv:1907.11692.

Radford et al. 2021. *Learning transferable visual models from natural language supervision* (CLIP). ICML.

Datasets

Zhai et al. 2019. *A large-scale study of representation learning with the visual task adaptation benchmark* (VTAB). arXiv:1910.04867.

Zhong, Xiong & Socher 2017. *Seq2SQL* (WikiSQL). arXiv:1709.00103.

Williams, Nangia & Bowman 2018. *A broad-coverage challenge corpus for sentence understanding through inference* (MNLI). NAACL.

Wang et al. 2019. *GLUE: a multi-task benchmark and analysis platform for natural language understanding.* ICLR.

Parkhi, Vedaldi, Zisserman & Jawahar 2012. *Cats and dogs* (Oxford Pets). CVPR.

Jaeger et al. 2014; Moreira et al. 2012; Wang et al. 2017. Shenzhen chest X-ray, INbreast and ChestX-ray14, used by MeLo.